

mcr_camera_2026base プログラム設計仕様書

GR-PEACH (RZ/A1H) マイコンカラーリー カメラクラス開発

バージョン 2.0

作成日 2026 年 8 月 13 日

作成者 Shiozawa

ステータス ドラフト

本文書は mcr_camera_2026base プロジェクトのプログラム設計仕様書です。
ベアメタル環境における割り込み駆動アーキテクチャの詳細設計を記述します。

改訂履歴

版	日付	著者	変更内容
1.0	2026-02-19	Shiozawa	初版作成
1.1	2026-02-24	Shiozawa	Onboard クラス・1ms タイマーの実装
1.2	2026-02-24	Shiozawa	オンボードクラスのインスタンス化・GPIO 初期化修正
1.3	2026-02-24	Shiozawa	GIC 割り込みディスパッチャ (IRQ) の実装
1.4	2026-02-24	Shiozawa	CPU ベクタベースアドレス (VBAR) の初期化追加
1.5	2026-02-24	Shiozawa	割り込み動作の根本原因 (Cortex-A9) 修正
1.6	2026-02-24	Shiozawa	OSTM タイマー周期がハードに反映されない問題の修正
1.7	2026-02-24	Shiozawa	タイマー割り込みが 1 回しか発生しない不具合の修正
1.8	2026-02-24	Shiozawa	個別割り込みハンドラからの不正リターン命令修正
1.9	2026-02-24	Shiozawa	IModule インターフェース実装・Onboard グローバル化
1.10	2026-02-24	Shiozawa	Onboard の setLed を setUserLed/setColorLed に分割
1.11	2026-02-24	Shiozawa	シリアル通信 (Serial) クラスの実装
1.12	2026-02-26	Shiozawa	LaTeX コンパイル用 DevContainer 環境の統合
1.13	2026-03-06	Shiozawa	SPEC.md の内容を TeX に統合・README.md 新規作成
1.14	2026-05-03	Shiozawa	RunController クラス (メイン走行制御) 追加。エンコーダ距離制御をタイマー (ms 単位) に置換。
2.0	2026-08-13	Shiozawa	実装準拠への全面改訂。 EMA (Embedded-Module-Architecture) 化を反映し、IModule を updateInput/updateOutput 版に更新。SystemData / ProjectConfig / ModuleTimer を新規記述。実在しない Trace 章を LineDetector 章へ、RunController 章を RunLogic 章へ差し替え。LineDetector / SDLogger / 起動シーケンス / パラメータ調整ガイドを追加。mcr_camera_2026base_SPEC.md の技術知見を「ベアメタル固有の実装知見」章と付録「不具合と対策の記録」へ移植し、同ファイルを廃止。

目次

改訂履歴	1
1 概要	4
1.1 用語定義	4
2 背景・課題	4
3 目的	5
4 システムアーキテクチャ	5
4.1 実行フロー図	5
4.2 3 フェーズモデル	6
4.3 共有データ構造 SystemData	6
4.4 ModuleTimer — ノンブロッキング時間計測	7
4.5 設定注入と ProjectConfig	7
5 ファイル構成	8
5.1 ディレクトリ構造	9
5.2 ディレクトリ役割	10
5.3 全ファイル一覧	10
5.4 機能とファイルの対応表	11
6 技術スタック	11
6.1 ビルド検証環境	11
7 API 仕様	13
7.1 IModule インターフェース	13
7.2 Encoder クラス	15
7.3 Camera クラス	17
7.4 LineDetector クラス	20
7.5 Motor クラス	22
7.6 Servo クラス	25
7.7 Onboard クラス	26
7.8 Serial クラス	29
7.9 SDCard クラス	31
7.10 SDLogger クラス	34
7.11 RunLogic — 走行ロジック	36
8 実行モデル詳細	41
8.1 起動シーケンス	41
8.2 3 フェーズ割り込みハンドラ	42
8.3 メインループと動作モード	43
8.4 出力反映のデータフロー	44

9	パラメータ調整ガイド	44
9.1	RUN_CONFIG — 走行ロジック	44
9.2	MOTOR_CONFIG / SERVO_CONFIG — アクチュエータ	46
9.3	LINE_DETECTOR_CONFIG — ライン検出	46
9.4	その他の Config	47
10	ベアメタル固有の実装知見	48
10.1	XIP 実行と命令フェッチ制約	48
10.2	MMU / L1 キャッシュの有効化	48
10.3	グローバル C++ コンストラクタの手動実行	49
10.4	ベアメタル向けダミーシンボル	50
10.5	GIC 割り込みディスパッチと VBAR	50
10.6	OSTM0 の設定手順	51
10.7	SCIF2 ボーレートのクロックソース	51
10.8	VDC5 / DVDEC の初期化	52
10.9	generate/ の手動修正箇所	52
10.10	DevContainer によるドキュメントビルド	53
付録 A	不具合と対策の記録	53
A.1	起動・割り込み系	53
A.2	シリアル通信系	54
A.3	カメラ系	54

1 概要

GR-PEACH (RZ/A1H) を使用した、マイコンカーラリー (MCR) カメラクラス開発のためのベースプログラムである。

▷ 設計思想

- mbed OS を使用せず、ルネサス標準の `iodefine.h` を用いたベアメタル (レジスタ直接操作) 環境で構築する
- 既存のスパゲッティコードを廃し、EMA (Embedded-Module-Architecture) 準拠のモジュール分割を採用する。全モジュールは共通インターフェース `IModule` を実装し、`init()` / `updateInput()` / `updateOutput()` の3点で統一的に扱う
- モジュール間のデータ受け渡しは共有構造体 `SystemData` に一本化し、モジュールが他モジュールのメソッドを直接呼ぶことを禁止する
- 調整対象の定数は `src/core/ProjectConfig.h` に集約し、「走行調整のためにドライバのソースを触る」状況をなくす

1.1 用語定義

用語	定義
MCR	マイコンカーラリーの略称
GR-PEACH	ルネサス RZ/A1H 搭載の開発ボード
MTU2	マルチファンクション・タイマ・ユニット 2。PWM 出力・位相計数に使用
VDC5	ビデオディスプレイコントローラ 5。DVDEC と組み合わせて NTSC 映像を取り込む
EMA	Embedded-Module-Architecture。IModule / SystemData / {Module}Config の3点セットでモジュールを構成する設計方針
IModule	全モジュール共通の基底インターフェース (§ 7.1)
SystemData	全モジュールの入出力データを集約する共有構造体 (§ 4.3)
Config	モジュールごとの設定構造体。実値は ProjectConfig.h で一括定義する (§ 9)
ModuleTimer	1ms カウンタ <code>g_timer_1ms</code> を時刻源とするノンブロッキングタイマー (§ 4.4)
ラッチ方式	出力値をいったん内部バッファに保持し、出力フェーズでのみハードウェアへ一括反映する方式
XIP	eXecute In Place。SPI フラッシュ上のコードを RAM へ展開せず直接実行する方式 (§ 10.1)

2 背景・課題

△ 開発環境の課題

1. **mbed OS サービス終了**: mbed のサービス終了に伴い、将来的な継続利用や保守が困難になる
2. **開発環境の老朽化**: 既存プロジェクトの e2 studio 環境やコンパイラバージョンが古く、最新の環境でビルドを通す必要がある
3. **ブラックボックス化の解消**: OS 内部処理への依存を減らし、デバッグを容易にする必要がある

3 目的

- 1. mbed OS に依存しない、RZ/A1H 用の C++ ベースプログラム環境の構築
- 2. クラス化・モジュール化による可読性・保守性の向上
- 3. 共通インターフェース (IModule) による統一的な制御フローの確立

4 システムアーキテクチャ

システムは OSTM0 による 1ms 周期のタイマー割り込みで同期して動作する。割り込みハンドラ `ostm0_interrupt_callback()` が Input → Logic → Output の 3 フェーズを毎周期実行し、メインループはフレーム完了待ち・ライン検出・ログ処理・デバッグ表示を担当する。

4.1 実行フロー図

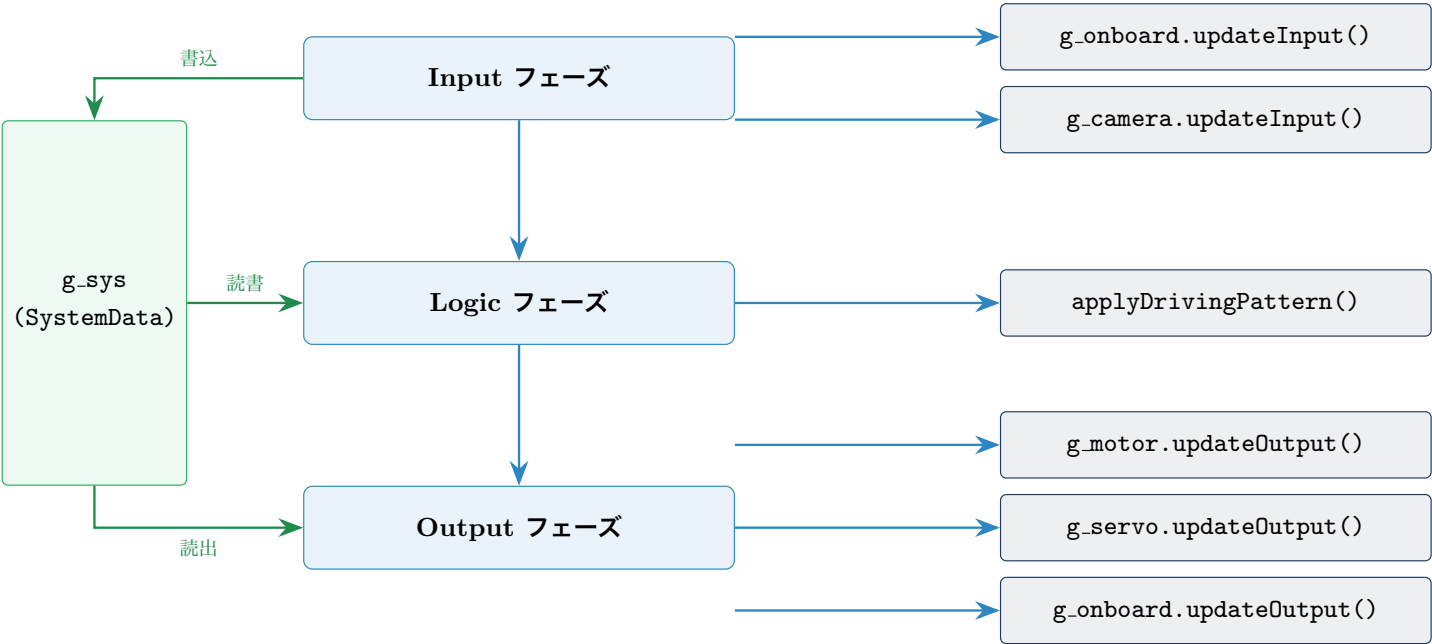


図 1: 1ms 割り込みハンドラの 3 フェーズ実行フロー

Input / Output の各フェーズは IModule* の配列 (`s_inputModules[]` / `s_outputModules[]`) をループして呼び出す。モジュールの `enabled` が `false` の間はスキップされるため、初期化に失敗したモジュールを配列から外さずに無効化できる。

```
1 static IModule* s_inputModules[] = {
2     &g_onboard,    // Input : SW 状態を sys.ob.sw へ
3     &g_camera,    // Input : 1ms ごとのフレーム段階処理
4 };
5 static IModule* s_outputModules[] = {
6     &g_motor,      // Output : sys.mot.*Cmd -> PWM
7     &g_servo,     // Output : sys.srv.angleCmd -> PWM
8     &g_onboard,   // Output : sys.ob.*LedCmd -> GPIO
9 };
10
11 for (int i = 0; i < N_IN; ++i) {
```

```

12     if (s_inputModules[i]->enabled) s_inputModules[i]->updateInput(g_sys);
13 }

```

Listing 1: Input / Output フェーズの配列駆動 (mcr_camera_2026base.cpp)

4.2 3 フェーズモデル

順序	フェーズ	内容
1	Input	ハードウェア → SystemData。各モジュールの updateInput(sys) がセンサ値を sys.* へ書き込む
2	Logic	SystemData → SystemData。applyDrivingPattern(sys) が状態機械を進め、出力指示を sys.mot.*Cmd / sys.srv.angleCmd / sys.ob.*LedCmd へ代入する（ドライバは呼ばない）
3	Output	SystemData → ハードウェア。各モジュールの updateOutput(sys) が *Cmd を読み、レジスタへ反映する

▷ 3 フェーズ分離の利点

Logic フェーズはハードウェアに一切触れない純粋な計算になるため、SystemData を差し替えるだけで PC 上の単体テストに載せられる。また出力タイミングが Output フェーズに集約されるので、1ms 内で出力が二度書きされたり、Logic の途中経過が物理出力に漏れることがない。

△ 全モジュールがラッチ方式ではない

Output フェーズ内での反映方法はモジュールによって異なる。Onboard は 4 個の LED 状態を内部バッファ ledState_[] にまとめてから GPIO へ一括書き込みする真のラッチ方式だが、Motor / Servo は updateOutput() 内で MTU2 レジスタへ直接書き込む。いずれも「Output フェーズでのみ書く」点は共通であり、Logic フェーズの途中経過が出力に漏れない性質は保たれている。

4.3 共有データ構造 SystemData

SystemData (src/core/SystemData.h) は全モジュールの入出力データを 1 つに集約した構造体である。実体はグローバル変数 g_sys (mcr_camera_2026base.cpp で定義) 1 個のみで、3 フェーズの全呼び出しがこの参照を受け取る。

```

1 struct SystemData {
2     CameraData      cam;    // Camera が所有
3     LineDetectorData line;  // LineDetector が所有
4     MotorData       mot;    // Motor が所有
5     ServoData       srv;    // Servo が所有
6     OnboardData     ob;     // Onboard が所有
7     EncoderData     enc;    // Encoder が所有
8     SerialData      ser;    // Serial が所有
9     SDLoggerData    sd;     // SDLogger が所有
10    RunData          run;    // RunLogic が所有
11 };
12 extern SystemData g_sys;

```

Listing 2: SystemData の定義

メンバ	型	主な内容
cam	CameraData	frameReady, frameCount, field, imageBufferPtr
line	LineDetectorData	deviation[120], crossLine, leftLine, rightLine, centerLine, sensorBin
mot	MotorData	leftCmd, rightCmd, leftActual, rightActual
srv	ServoData	angleCmd, angleActual
ob	OnboardData	sw, ledRedCmd, ledGreenCmd, ledBlueCmd, userLedCmd
enc	EncoderData	totalCount, magaCount, cnt, avgCnt, filteredCnt
ser	SerialData	txCount
sd	SDLoggerData	ready, logCount, full, saveRequested, saveDone
run	RunData	pattern, handleVal, finished, 3 本の ModuleTimer

▷ データの所有権

{Module}Data 構造体の宣言は各モジュールのヘッダ側に置き、SystemData.h はそれを include して集約するだけである。各モジュールは原則として**自身の Data だけ**を読み書きし、他モジュールのデータを横断参照してよいのは Logic フェーズの関数だけ、というルールで運用する。

△ 画素バッファは SystemData に置かない

160×120 の輝度バッファ (約 19KB) と YCbCr422 中間バッファ (約 38KB) は Camera クラス内に実体を持ち、SystemData からは cam.imageBufferPtr 経由で参照する。巨大な配列を SystemData に持たせるとメモリ効率とコピーコストが悪化するため。

4.4 ModuleTimer — ノンブロッキング時間計測

ベアメタル環境には Arduino の millis() に相当する API がないため、OSTM0 の 1ms 割り込みでインクリメントされるグローバルカウンタ volatile unsigned long g_timer_1ms を時刻源とする軽量クラスを用意している (src/core/ModuleTimer.h)。

メソッド	機能
void setTime()	現在時刻を起点としてタイマーを開始する
unsigned long getNowTime() const	setTime() からの経過時間 [ms]。未起動なら 0
bool isExpired(unsigned long ms) const	指定時間以上経過したか。未起動なら false
void reset()	停止状態に戻す
bool isRunning() const	起動中かどうか

RunData はこれを 3 本保持し、旧実装のカウンタ変数を置き換えている (§ 7.11)。

▷ 待ちループを書かないこと

ModuleTimer は「経過したか」を問い合わせるだけで、内部でブロックしない。1ms 割り込みの中で while (!timer.isExpired(n)) のような待機を書くと g_timer_1ms が進まなくなり、確実にハングする。

4.5 設定注入と ProjectConfig

各モジュールは {Module}Config 構造体を **コンストラクタ引数で受け取り**、内部に _config としてコピーして保持する。実値の定義は src/core/ProjectConfig.h の 1 ファイルに集約されており、走行パラメータの調整でドラ

イバのソースを編集する必要はない。

```
1 Serial      g_serial      (SERIAL_CONFIG);
2 Onboard     g_onboard     (ONBOARD_CONFIG);
3 Motor       g_motor       (MOTOR_CONFIG);
4 Servo       g_servo       (SERVO_CONFIG);
5 Encoder     g_encoder     (ENCODER_CONFIG); // 現状 init呼出は未接続/
6 Camera      g_camera      (CAMERA_CONFIG);
7 LineDetector g_lineDetector(LINE_DETECTOR_CONFIG);
8 SDCard      g_sdcard      (SDCARD_CONFIG);
9 SDLogger    g_sdlogger    (SDLOGGER_CONFIG);
```

Listing 3: Config 注入によるインスタンス生成 (mcr_camera_2026base.cpp)

実際の設定値は [§ 9](#)「パラメータ調整ガイド」にまとめる。

△ Config はグローバル ctor に依存する

`_config` メンバはコンストラクタ初期化で埋まる。本プロジェクトの `start.S` は `__libc_init_array()` を呼ばないため、`main()` 冒頭の `runGlobalConstructors()` を経由しないと `_config` が BSS のゼロのままになり、サーボ出力が 0 になる等の致命的な不具合になる。詳細は [§ 10.3](#) を参照。

5 ファイル構成

5.1 ディレクトリ構造

```
mcr_camera_2026base/  
  src/  
    mcr_camera_2026base.cpp ← メイン・GIC/OSTM0 初期化・割り込みハンドラ  
  core/  
    IModule.h ← 基底インターフェース  
    SystemData.h ← 全モジュール共有データ  
    ProjectConfig.h ← 全パラメータの実値定義  
    ModuleTimer.h ← ノンブロッキングタイマー  
  logic/  
    RunLogic.h / RunLogic.cpp ← 走行状態機械（純関数）  
  drivers/  
    Camera.h / Camera.cpp ← NTSC キャプチャ (VDC5+DVDEC)  
    LineDetector.h / .cpp ← ライン検出・偏差算出  
    Encoder.h / Encoder.cpp ← エンコーダ制御  
    Motor.h / Motor.cpp ← モータ制御  
    Servo.h / Servo.cpp ← サーボ制御  
    Onboard.h / Onboard.cpp ← LED・スイッチ制御  
    Serial.h / Serial.cpp ← シリアル通信  
    SDCard.h / SDCard.cpp ← SD カード SPI ブロックドライバ  
    SDLogger.h / SDLogger.cpp ← ログバッファ + CSV 書き出し  
  fatfs/ ← ChaN's FatFs R0.11a + diskio ブリッジ  
  video/ ← VDC5/DVDEC ドライバ群 (DisplayBase API)  
  system/  
    system_init.c ← FPU/MMU/L1 キャッシュ有効化  
    mmu_rzA1H.c ← MMU 翻訳テーブル生成  
  generate/ ← e2 studio 自動生成（一部手動修正あり）  
    iodef.h ← レジスタ定義  
    inthandler.c ← 割り込みディスパッチ ★手動修正  
    start.S / linker_script.ld ← 起動コード・リンクスクリプト ★手動修正  
  tests/ ← PC 上の単体テスト (CMake + レジスタモック)  
  doc/  
    main.tex / main.pdf ← 本仕様書  
  .devcontainer/ ← LaTeX ビルド用 DevContainer
```

5.2 ディレクトリ役割

ディレクトリ	役割
src/core/	システム基盤 (IModule, SystemData, ProjectConfig, ModuleTimer)
src/logic/	Logic フェーズの走行ロジック (RunLogic)。ハードウェアに触れない
src/drivers/	ハードウェアドライバ (Camera, LineDetector, Motor, Servo, Encoder, Onboard, Serial, SDCard, SDLogger)
src/drivers/fatfs/	ChaN's FatFs R0.11a と SDCard へのブリッジ (diskio.cpp)
src/drivers/video/	mbed-gr-libs 由来の VDC5/DVDEC ドライバ群と DisplayBase API
src/system/	FPU / MMU / L1 キャッシュ初期化 (mbed OS の SystemInit() 移植)
generate/	e2 studio 自動生成ファイル (§ 10.9 の手動修正あり)
tests/	PC 上で動かす単体テスト (レジスタをモック化して Motor/Servo/Encoder を検証)
doc/	本仕様書 (main.tex / main.pdf)
.devcontainer/	LaTeX ドキュメントビルド用のコンテナ環境定義

5.3 全ファイル一覧

ファイル	役割
src/mcr_camera_2026base.cpp	メインエントリ・GIC/OSTM0 初期化・3 フェーズ割り込みハンドラ・2つの動作モードループ
src/core/IModule.h	全モジュール共通の基底インターフェース
src/core/SystemData.h	全モジュールの Data を集約した共有構造体
src/core/ProjectConfig.h	全 *_CONFIG インスタンスの実値定義 (調整の入口)
src/core/ModuleTimer.h	g_timer_1ms を時刻源とするノンブロッキングタイマー
src/logic/RunLogic.h/.cpp	走行状態機械 (applyDrivingPattern())・RunPattern 定義
src/drivers/Camera.h/.cpp	NTSC 160×120 キャプチャ (VDC5 ch0 + DVDEC0)
src/drivers/LineDetector.h/.cpp	偏差算出・クロス/ハーフ/センターライン検出・8 点センサ生成
src/drivers/Motor.h/.cpp	モータ PWM 制御 (MTU2 ch3/4 リセット同期 PWM)
src/drivers/Servo.h/.cpp	サーボ PWM 制御 (MTU2 ch0 PWM モード 1)
src/drivers/Encoder.h/.cpp	ロータリーエンコーダ読取 (MTU2 ch1 位相計数モード 2)
src/drivers/Onboard.h/.cpp	オンボード LED・スイッチ制御 (GPIO ポート 6)
src/drivers/Serial.h/.cpp	デバッグシリアル通信 (SCIF2, 230400bps)
src/drivers/SDCard.h/.cpp	SD カード SPI ブロックドライバ (RSPI2)。FatFs の下位層
src/drivers/SDLogger.h/.cpp	走行ログのバッファリングと CSV 書き出し
src/drivers/fatfs/ff.c	FatFs R0.11a 本体
src/drivers/fatfs/diskio.cpp	FatFs → SDCard のディスク I/O ブリッジ
src/drivers/video/	VDC5/DVDEC ドライバ群と DisplayBase API (40 ファイル超)
src/system/system_init.c	FPU / MMU / L1 キャッシュ有効化
generate/inthandler.c	IRQ ディスパッチャと OSTM0 コールバック呼び出し (手動修正)

ファイル	役割
generate/linker_script.ld	.init_array 収集を追加したリンクスクリプト（手動修正）
generate/start.S	起動コード（VBAR 設定・IRQ スタック確保を追加、手動修正）

5.4 機能とファイルの対応表

機能	メインファイル	関連ファイル
3 フェーズ割り込み	mcr_camera_2026base.cpp	IModule.h, SystemData.h, inthandler.c
走行状態機械	RunLogic.cpp	SystemData.h, ProjectConfig.h, ModuleTimer.h
ライン検出	LineDetector.cpp	Camera.h, SystemData.h, ProjectConfig.h
カメラ入力	Camera.cpp	video/DisplayBace.h, video/gr_peach_vdc5.c
モータ制御	Motor.cpp	IModule.h, iodefne.h
サーボ制御	Servo.cpp	IModule.h, iodefne.h
エンコーダ	Encoder.cpp	IModule.h, iodefne.h
LED・スイッチ	Onboard.cpp	IModule.h, iodefne.h
シリアル通信	Serial.cpp	IModule.h, iodefne.h
走行ログ	SDLogger.cpp	SDCard.cpp, fatfs/ff.c, fatfs/diskio.cpp
起動時初期化	system_init.c	mmu_rzA1H.c, start.S

6 技術スタック

項目	内容
ターゲットボード	GR-PEACH (ルネサス RZ/A1H, ARM Cortex-A9)
開発環境	Renesas e2 studio (ビルド構成 HardwareDebug)
言語	C++ (ベアメタル、mbed OS 不使用)。ProjectConfig.h が inline const を使うため C++17 以上が必要
コンパイラ	GCC for Renesas ARM (arm-none-eabi-gcc/g++ 13.3.1)。確認済みの組み合わせは S 6.1
浮動小数点	VFPv3-D16, hard ABI, リトルエンディアン
実行方式	XIP (SPI フラッシュ直接実行) + MMU / L1 キャッシュ有効 (S 10.1)
レジスタ定義	iodefine.h (e2 studio 自動生成)
割り込み	GIC (Generic Interrupt Controller) + OSTM0 による 1ms インターバル
映像ドライバ	mbed-gr-libs 由来の VDC5/DVDEC ドライバ + DisplayBase API
ファイルシステム	ChaN's FatFs R0.11a (diskio.cpp 経由で SDCard に接続)
ドキュメント	L ^A T _E X(uplatex + dvipdfmx, latexmk)。DevContainer で環境統一

6.1 ビルド検証環境

ファームウェアのビルドが通ることを確認済みの環境を記録する。組み込み開発ではツールチェーンのバージョン差でビルドが通らなくなることがあるため、別環境で再構築する際は**まずこの組み合わせを再現すること**を勧める。

項目	確認済みの値
IDE	Renesas e2 studio バージョン: (確認中 — 要記入)
ビルド構成	HardwareDebug
ツールチェーン	GNU Arm Embedded Toolchain (Arm GNU Toolchain arm-none-eabi 13.3 rel1)
.cproject の登録値	toolchain.id = gcc-arm-embedded, toolchain.version = 13.3.1.arm-13-24
コンパイラ	arm-none-eabi-gcc / arm-none-eabi-g++ 13.3.1 (生成 makefile の GCC_VERSION=13.3.1 と一致)
CDT ツールチェーン	GCC for Renesas RZ (com.renesas.cdt.managedbuild.gcc.rz.toolchain.debug.update)

6.1.1 ターゲット設定

.cproject に登録されているビルドオプションと、実際に生成されたコンパイルコマンドの内容は次のとおり。

項目	設定値	コンパイラオプション
アーキテクチャ	ARM (32bit)	-- (architecture.arm)
CPU	Cortex-A9	-mcpu=cortex-a9
命令セット	ARM	-marm
エンディアン	リトルエンディアン	-mlittle-endian
FPU	VFPv3-D16	-mfpu=vfpv3-d16
Float ABI	hard	-mfloat-abi=hard
最適化レベル	なし (-O0)	-O0
デバッグ情報	DWARF 4	-g -gdwarf-4

上記に加えて次のオプションが付与される。

```
-fsigned-char -ffunction-sections -fdata-sections -fno-strict-aliasing
-fabi-version=0 -fmessage-length=0 -fdiagnostics-parseable-fixits
-Wnull-dereference -Wstack-usage=100
```

▷ 値の出典

- 本節の値は次の実測に基づく。
- ツールチェーンとターゲット設定: リポジトリ直下の .cproject (com.renesas.cdt.managedbuild.core.toolchainInfo および各 <option> 要素)
 - コンパイラのバージョンとオプション列: git 履歴に残る HardwareDebug/ のビルド生成物 (makefile.init の GCC_VERSION、*.o.in のコンパイルコマンド)。現在 HardwareDebug/ は .gitignore で除外されている

△ e2 studio の製品バージョンは未確定

e2 studio 本体のバージョンを特定できる情報は、リポジトリ内のどのファイルにも記録されていない (.settings/e2studio-project.prefs が持つのは構成 ID のみ)。そのため上表の該当欄は**未記入のまま**である。

バージョンが判明したら、プレアンブルの \newcommand{\ideversion}{...} 1 箇所を書き換えるだけで本表に反映される。e2 studio 側での確認方法は「ヘルプ → e2 studio について」。

7 API 仕様

本章では各モジュールの公開 API をデータシート形式で記述する。src/drivers/ 配下の全クラスは IModule インターフェース (§ 7.1) を実装する。Logic レイヤーの RunLogic (§ 7.11) だけはクラスではなく自由関数の集合であり、IModule を継承しない。

▷ 定数値の扱い

各クラスの static const (Motor::MAX_POWER 等) は後方互換のために残されているもので、**実行時に使われるのは _config 経由の値**である。実値の一覧は § 9 を参照すること。

7.1 IModule インターフェース

IModule — 基底インターフェース

ファイル: src/core/IModule.h

種別: 純粋仮想クラス

概要

全てのドライバが実装する共通インターフェース。3 フェーズ実行モデル (§ 4.2) に対応し、入力と出力を別メソッドに分けている点が特徴である。

- 入力専用モジュール (Camera, Encoder) は updateInput() のみを override する
- 出力専用モジュール (Motor, Servo, Serial, SDCard, SDLogger) は updateOutput() のみを override する
- 入出力両方のモジュール (Onboard) は両方を override する

SystemData は前方宣言のみを行い、各モジュールヘッダから SystemData.h を include させないことで循環依存を避けている。

関数ドキュメント

◆ init()

```
virtual bool init() = 0
```

初期化処理。レジスタ設定・変数初期化を行う。システム起動時に main() から 1 回のみ呼び出される。

戻り値: bool — true=成功 / false=失敗。失敗時は呼び出し側で enabled = false に落とす運用を推奨する。

◆ updateInput(sys)

```
virtual void updateInput(SystemData& sys)
```

入力フェーズ。ハードウェアを読み取り、結果を sys の自モジュール領域へ書き込む。デフォルトは空実装のため、出力専用モジュールは override 不要。

◆ updateOutput(sys)

virtual void updateOutput(SystemData& sys)

出力フェーズ。sys の指示値 (*Cmd) を読み、ハードウェアへ反映する。デフォルトは空実装のため、
入力専用モジュールは override 不要。

◆ deinit()

virtual void deinit()

終了処理 (バス停止、リソース解放等)。デフォルトは何もしない。

メンバ変数

変数名	型	初期値	説明
enabled	bool	true	false の間は呼び出し側の配列ループが updateInput/updateOutput をスキップする

インターフェース定義

```
1 struct SystemData;    // 前方宣言 (循環依存の回避)
2
3 class IModule {
4 public:
5     virtual ~IModule() {}
6
7     // 初期化処理 (成功 = 、失敗 true = ) false
8     virtual bool init() = 0;
9
10    // 入力フェーズ: ハードウェア -> SystemData
11    virtual void updateInput(SystemData& sys) { (void)sys; }
12
13    // 出力フェーズ: SystemData -> ハードウェア
14    virtual void updateOutput(SystemData& sys) { (void)sys; }
15
16    // 終了処理 (バス停止、リソース解放等)
17    virtual void deinit() {}
18
19    // false の間は updateInput / updateOutput をスキップする
20    bool enabled = true;
21 };
```

Listing 4: IModule インターフェース定義

新規モジュールの追加手順

- src/drivers/Xxx.h に XxxConfig と XxxData を宣言する

2. class Xxx : public IModule を定義し、explicit Xxx(const XxxConfig&) を用意する

3. SystemData.h に XxxData xxx; を追加し、ヘッダを include する

4. ProjectConfig.h に XXX_CONFIG の実値を定義する

5. mcr_camera_2026base.cpp でグローバルインスタンス g_xxx を生成し、main() で init() を呼び、必要なフェーズの配列へ登録する

7.2 Encoder クラス

Encoder — エンコードドライバ

ファイル: src/drivers/Encoder.h, Encoder.cpp

継承: IModule

Input フェーズ

概要

MTU2 チャンネル 1 の位相計数モード 2 を使用し、エンコーダのパルスをカウントする。1ms ごとに MTU2TCNT_1 を読んでゼロクリアし、その差分 (1ms あたりのパルス数) を累積カウンタと 2 種類のフィルタに通す。

⚠ 現状 ISR に未接続

g_encoder のインスタンス自体は生成されているが、main() での init() 呼び出しと s_inputModules[] への登録はまだ行われていない。したがって sys.enc.* は常に 0 のままである。SDLogger 連携が必要になった時点で有効化する予定。

機能一覧

- MTU2 位相計数モード 2 による前進/後退を区別したパルス計数
- 走行距離換算用の累積カウンタ (totalCount) とクランク検出用 (magaCount) の 2 系統
- 移動平均フィルタ (段数 filterN)
- RC フィルタ (指数移動平均、係数 rcAlpha)

設定 (EncoderConfig)

メンバ	型	実値	説明
filterN	int	4	移動平均の段数 (上限は Encoder::FILTER_N = 4)
rcAlpha	float	0.5	RC フィルタ係数 α
aPinFmt	int	0	A 相 P1_0 (TCLKA)
bPinFmt	int	10	B 相 P1_10 (TCLKB)

出力データ (EncoderData / sys.enc)

メンバ	型	説明
totalCount	int	走行距離換算用の累積カウント
magaCount	int	クランク検出用の累積カウント
cnt	int	直近 1ms あたりのカウント (符号付き)
avgCnt	int	移動平均後のカウント
filteredCnt	float	RC フィルタ後のカウント

▷ 距離ではなくパルス数を扱う

本実装が保持しているのは**パルスカウント (int)**であり、距離 [mm] や速度への換算は行っていない。走行状態の遷移条件は経過時間 [ms] で管理する方針 (S 7.11) のため、距離換算係数 (1 パルスあたりの移動距離) は定義していない。

関数ドキュメント

◆ init()

bool init() override

MTU2 チャンネル 1 を位相計数モード 2 で初期化する。P1.0 / P1.10 のピン設定、MTU2 のスタンバイ解除、カウンタスタートを行う。

◆ updateInput(sys)

void updateInput(SystemData& sys) override

MTU2TCNT_1 を符号付き 16bit として読み出して 0 にリセットし、累積カウンタ・移動平均・RC フィルタを更新して sys.enc.* に格納する。RC フィルタは $rc \leftarrow \alpha \cdot rc + (1 - \alpha) \cdot cnt$ で更新する。

◆ clearTotal() / clearMaga()

void clearTotal() / void clearMaga()

それぞれ totalCount_ / magaCount_ を 0 にリセットする。

◆ 後方互換 getter

int getTotalCount() const / int getMagaCount() const / int getCnt() const / float
getFilteredCnt() const

内部状態を直接返す。getCnt() は移動平均後の値 (avg_) を返す点に注意。新規コードでは sys.enc.* を読むこと。

ハードウェアマッピング

信号	ピン	レジスタ	説明
A 相	P1_0 (TCLKA)	MTU2TCNT_1	位相計数モード 2 の入力
B 相	P1_10 (TCLKB)	MTU2TCNT_1	同上 (位相差で方向を判別)

7.3 Camera クラス

Camera — NTSC ビデオキャプチャドライバ

ファイル: src/drivers/Camera.h, Camera.cpp

継承: IModule

Input フェーズ

概要

VDC5 チャンネル 0 と DVDEC0 を用いて NTSC 映像を 160×120 の YCbCr422 として取り込み、輝度 (Y) 成分のみを抽出して 160×120 のグレースケールバッファに格納する。初期化には mbed-gr-lib 由来の DisplayBase API (Graphics_init → Graphics_Video_init → Video_Write_Setting) を使用する。

機能一覧

- VDC5 + DVDEC による NTSC インターレース映像のキャプチャ
- フレーム処理の 4 ステップ分割 (XIP 環境の 1ms 制約に対応)
- Vfield 割り込みによるフィールド同期
- 座標指定の輝度取得 (getPixel) と 8 点センサ変換 (thresholdConvert)

設定 (CameraConfig)

メンバ	実値	説明
pixelWidth	160	水平ピクセル数
pixelHeight	120	垂直ピクセル数
thresholdDefault	170	8 点センサ取得時のデフォルト閾値
thresholdRow	60	8 点センサ取得行
thresholdDiff	8	8 点センサ閾値の差分

出力データ (CameraData / sys.cam)

メンバ	型	説明
frameReady	bool	新フレーム完了フラグ。メインループが消費後に false へ戻す
frameCount	uint32_t	累積フレーム数 (デバッグ表示用)
field	int	0=トップ / 1=ボトムフィールド
imageBufferPtr	const volatile unsigned char*	輝度バッファ (160×120) へのポインタ

フレーム処理の 4 ステップ分割

XIP 実行環境では `imageCopy()` 1 回に約 7-8ms を要し、1ms 割り込みに収まらない (§ 10.1)。そのため 1 フレームの処理を 4 ステップに分割し、1ms 割り込みごとに 1 ステップだけ進める。

ステップ	処理	内容
0	<code>imageCopy(0)</code>	Vfield 値を <code>capturedField_</code> にキャプチャし、前半 (0-59 行) をコピー
1	<code>imageCopy(1)</code>	後半 (60-119 行) をコピー
2	<code>extractBrightness(0)</code>	前半の Y 成分を抽出
3	<code>extractBrightness(1)</code>	後半の Y 成分を抽出。完了時に <code>frameReady_</code> を立てる
4 以降	Vfield 待ち	次の Vfield 発生まで即リターンし、メインループへ CPU 時間を返す

▷ `capturedField_` 方式が必要な理由

XIP 環境では 4 ステップの合計が NTSC の Vfield 間隔 (16.7ms) を超えるため、`imageCopy()` と `extractBrightness()` がそれぞれ Vfield トグルを読み直すと異なるフィールド値を使ってしまい画像が乱れる。ステップ 0 で 1 回だけ値を確保し、4 ステップ全体で同じ値を使うことでこれを防いでいる。また、フレーム処理中 (ステップ 0-3) は Vfield 変化を無視する。処理中にリセットを掛けるとステップ 1 以降へ進めず、輝度バッファが永久に更新されない。

関数ドキュメント

◆ `init()`

```
bool init() override
```

DisplayBase API で VDC5 / DVDEC を初期化し (NTSC 160×120 YCbCr422 入力)、メモリ書き込み設定を行ってビデオキャプチャを開始する。初期化中の Vsync / Vfield 待ちは割り込みベースで行うため、事前に GIC が有効化されている必要がある (§ 8.1)。

◆ `updateInput(sys)`

```
void updateInput(SystemData& sys) override
```

1ms ごとに呼ばれ、上表のステップを 1 つ進める。毎周期の末尾で `sys.cam.*` を最新状態に更新する。メインループが `sys.cam.frameReady` を `false` に戻すと内部状態もリセットされる。

◆ `getPixel(x, y)`

```
unsigned char getPixel(int x, int y) const
```

輝度バッファから座標 (x,y) のピクセル値を返す。

引数: `x` (0-159), `y` (0-119) 戻り値: `unsigned char` — 輝度 (0-255)

◆ getImageBuffer()

```
const volatile unsigned char* getImageBuffer() const
```

輝度バッファ先頭への読み取り専用ポインタを返す。

◆ thresholdConvert(gyou, threshold, diff)

```
unsigned char thresholdConvert(int gyou, int threshold, int diff) const
```

参考プロジェクトの shikiichi_henkan 相当。指定行の 8 点 ($x = 31, 43, 54, 71, 88, 105, 116, 128$) の輝度を取得し、閾値を自動調整して 2 進数 8 ビットに変換する。

引数: gyou — 行番号 (0-119), threshold — 閾値, diff — 差分閾値

戻り値: unsigned char — 8 点センサ値

◆ vfieldCallback / vsyncCallback [static]

```
static void vfieldCallback(DisplayBase::int_type_t) / static void  
vsyncCallback(DisplayBase::int_type_t)
```

VDC5 の Vfield / Vsync 割り込みから呼ばれ、内部のフィールドトグルとカウンタを更新する。フレーム同期の起点となる。

内部変数 (抜粋)

変数名	型	説明
frameStep_	int	フレーム処理ステップカウンタ (0-4)
fieldToggle_	volatile int	トップ/ボトムフィールドのトグル
capturedField_	int	ステップ 0 で確保したフィールド値
imageBuffer_	volatile unsigned char[19200]	輝度データ (最終出力)
ycbcrBuffer_	unsigned char[38400]	YCbCr422 中間バッファ

△ 廃止された API

旧仕様の isFrameReady() / clearFrameReady() は**廃止済み**である。フレーム完了の検知は sys.cam.frameReady を直接読み書きする方式に変更された。また、旧仕様に記載されていた画像処理 API (resize, binarize, percentileMethod, discriminantAnalysis, extractPart, standardDeviation, covariance, matchPattern) は**いずれも実装されていない**。ライン検出は LineDetector (S 7.4) が担当する。

7.4 LineDetector クラス

LineDetector — ライン検出モジュール

ファイル: src/drivers/LineDetector.h, LineDetector.cpp

継承: IModule

Input フェーズ (メインループ実行)

概要

Camera が生成した輝度バッファからコースの白線を検出し、各行の偏差・各種ラインフラグ・8 点センサ値を `sys.line.*` に格納する。PID 制御は行わず、ハンドル角の算出は RunLogic 側の `calcHandle()` (比例ゲイン 3 段切替) が担当する。

△ ISR ではなくメインループから呼ぶ

処理が `HEIGHT × WIDTH` のループで重いため、1ms 割り込み内では呼ばない。メインループが `sys.cam.frameReady == true` を検出したときにのみ `g_lineDetector.updateInput(g_sys)` を実行する。このため `s_inputModules[]` 配列にも登録されていない。

▷ g_camera を直接呼ぶ意図的な例外

本モジュールは `sys.cam.imageBufferPtr` を経由せず、`g_camera.getPixel()` と `g_camera.thresholdConvert()` を**直接呼ぶ**。§ 4.3 の「他モジュールのデータを参照してよいのは Logic フェーズのみ」というルール of 例外だが、1 フレームあたり数万回の画素アクセスが発生するため、性能上の理由で**意図的にこの経路を採っている**。

機能一覧

- 全 120 行分の偏差算出 (`deviation[]`)
- クロスライン / 左ハーフライン / 右ハーフライン / センターラインの検出
- 8 点センサ値 (`sensorBin`) の生成
- 走行パターン (`sys.run.pattern`) に応じた検出パラメータの自動切替

クラス定数

定数名	値	説明
WIDTH	160	画像幅
HEIGHT	120	画像高さ (<code>deviation[]</code> の要素数と一致させること)
CENTER	80	画像中心の <i>x</i> 座標。偏差 0 の基準

出力データ (LineDetectorData / sys.line)

メンバ	型	説明
deviation[120]	int	各行の偏差。正＝ラインが左寄り、負＝ラインが右寄り
crossLine	bool	クロスライン検出フラグ
leftLine	bool	左ハーフライン検出フラグ
rightLine	bool	右ハーフライン検出フラグ
centerLine	bool	センターライン検出フラグ
sensorBin	unsigned char	8 点センサ値（ビット列）
detectRow	int	直近で使用された検出行（デバッグ表示用）

パターン別パラメータの切替

sys.run.pattern の値に応じて、3 組のパラメータセット（LineDetectorPatternParams）を切り替える。

パラメータセット	適用条件	用途
patternNormal	pattern == 11 (RP_TRACE_NORMAL)	通常トレース時
patternStart	pattern == 1 または 2	スタートバー接近・待機時
patternOther	上記以外	クランク・レーンチェンジ等

各セットのメンバの意味は次のとおり。負値には特別な意味がある。

メンバ	意味
crosslineWidth	クロスライン判定に使う幅。< 0 なら実行時に detectRow_ + 20 を算出
detectHeight	検出に使う行の厚み
crossCountThreshold	クロスライン判定の画素数閾値。< 0 なら crosslineWidth との加算値
brightnessRatio	最大輝度に対する相対閾値の比率
brightnessAbs	> 0 なら絶対閾値として使用、= 0 なら brightnessRatio を使用
centerWidth	センターライン判定に使う幅

実値は [S 9.3](#) を参照。

関数ドキュメント

◆ init()

bool init() override

偏差配列とフラグ類をゼロクリアし、検出行・検出高さを Config の既定値へ戻す。グローバルコンストラクタが走らない環境への保険として、コンストラクタと同じ初期化をここでも明示的に行っている。

◆ updateInput(sys)

```
void updateInput(SystemData& sys) override
```

以下を順に実行し、結果を `sys.line.*` へ書き込む。

1. `sys.run.pattern` を `pattern_` へコピー（パラメータ切替に使用）
2. `calcDeviation()` — 全行の偏差を算出
3. `calcLineFlags()` — クロス/左右ハーフ/センターの各フラグを判定
4. `updateSensorBin()` — 8 点センサ値を生成

◆ getDeviation(row)

```
int getDeviation(int row) const
```

指定行の偏差を返す。範囲外 ($row < 0$ または $row \geq 120$) の場合は 0 を返す。

◆ フラグ getter

```
bool isCrossLine() const / isLeftLine() / isRightLine() / isCenterLine()
```

各ラインフラグを返す。デバッグ表示用の後方互換 API であり、走行ロジックからは `sys.line.*` を読むこと。

◆ getSensorBin() / sensorInput(mask)

```
unsigned char getSensorBin() const / unsigned char sensorInput(unsigned char mask)
const
```

8 点センサ値をそのまま、またはマスクを適用して返す。マスク定数は `MASK2_2(0x66)`, `MASK4_4(0xff)` 等がヘッダに定義されている。

7.5 Motor クラス

Motor — モータ PWM ドライバ

ファイル: `src/drivers/Motor.h`, `Motor.cpp`

継承: `IModule`

Output フェーズ

概要

MTU2 チャンネル 3/4 を使用したリセット同期 PWM モードで左右モータを制御する。出力指示は `sys.mot.leftCmd` / `rightCmd` に $-100 \sim +100$ [%] の整数で与え、`updateOutput()` が PWM デューティと方向ピンへ反映する。

機能一覧

- MTU2 リセット同期 PWM モードによる左右独立駆動
- 符号による正転/逆転の方向制御
- `maxPower` による出力上限クランプ

- 反映後の値を `sys.mot.*Actual` へ書き戻し (ログ・デバッグ用)

設定 (MotorConfig)

メンバ	実値	説明
pwmCycle	33332	PWM 周期カウンタ値 (1ms @ P0φ/1)
maxPower	70	出力上限 [%]。実効的な最大出力はこの値
leftPwmPinFmt	4	P4.4 (TIOC4A)
rightPwmPinFmt	5	P4.5 (TIOC4B)
leftDirPinFmt	6	P4.6 (GPIO)
rightDirPinFmt	7	P4.7 (GPIO)

▷ MAX_POWER と maxPower は別物

ヘッダの `Motor::MAX_POWER = 100` は後方互換のために残る `static const` であり、実行時のクランプに使われるのは `MOTOR_CONFIG.maxPower = 70` のほうである。また `RunLogic::calcMotorVal()` は通常走行時のモータ出力を `MOTOR_CONFIG.maxPower` そのものに設定するため、この 1 項目を変えると通常走行速度が直接変わる。

入出力データ (MotorData / sys.mot)

メンバ	型	方向	説明
leftCmd	int	Logic → Motor	左モータ指示値 [-100, +100]
rightCmd	int	Logic → Motor	右モータ指示値 [-100, +100]
leftActual	int	Motor → 外部	クランプ後に実際に反映した値
rightActual	int	Motor → 外部	同上

関数ドキュメント

◆ `init()`

`bool init() override`

P4.4/P4.5 をペリフェラル (PWM 出力) に、P4.6/P4.7 を GPIO 出力 (方向制御) に設定し、MTU2 のスタンバイを解除してチャネル 3/4 をリセット同期 PWM モードで起動する。周期レジスタ `MTU2TGRA_3` / `MTU2TGRC_3` に `_config.pwmCycle` を設定し、デューティは 0 で開始する。

◆ `updateOutput(sys)`

`void updateOutput(SystemData& sys) override`

`sys.mot.leftCmd` / `rightCmd` を内部の `set()` に渡して PWM へ反映し、クランプ後の値を `sys.mot.leftActual` / `rightActual` に書き戻す。

◆ getLeft() / getRight()

```
int getLeft() const / int getRight() const
```

内部に保持している現在の出力値を返す（デバッグ用）。

◆ set(left, right) [private]

```
void set(int left, int right)
```

両輪の値を clamp() で [-maxPower, +maxPower] に丸めてから applyLeft() / applyRight() を呼ぶ。

◆ applyLeft(val) / applyRight(val) [private]

```
void applyLeft(int val) / void applyRight(int val)
```

方向ピンを設定し、デューティ値を MTU2 レジスタへ書き込む。書き込み先は**バッファレジスタである MTU2TGRC_4 (左) と MTU2TGRD_4 (右) のみ**で、TGRC_4 / TGRD_4 は init() 時の初期化にしか使わない。

デューティ値の計算式:

$$duty = \frac{(pwmCycle - 1) \times |val|}{100}$$

ハードウェアマッピング

信号	ピン	レジスタ	説明
左モータ PWM	P4.4 (TIOC4A)	MTU2TGRC_4	デューティ (バッファレジスタ)
右モータ PWM	P4.5 (TIOC4B)	MTU2TGRD_4	デューティ (バッファレジスタ)
左モータ 方向	P4.6 (GPIO)	GPIO.P4 bit6	High: 前進 / Low: 後退 (極性反転あり)
右モータ 方向	P4.7 (GPIO)	GPIO.P4 bit7	Low: 前進 / High: 後退

△ 要実機確認: 左モータの方向ピン極性が左右で非対称

実装 (Motor::applyLeft()) では左モータのみ方向ピンの極性が反転しており、**前進時に P4.6 を High** にする。右モータ (applyRight()) は前進時に P4.7 を Low にする通常の極性である。

ソース上のコメントは「左モーターは配線/ギアの都合で正転=後退となっているため、ソフト側で方向ピンの極性を反転して補正する」と説明しており、**機体側の配線に合わせた意図的な補正**と読める。ただし本仕様書の旧版は左右とも「Low: 正転」と記述しており、記述が食い違っている。

本表は**現在の実装の実態**を記載したものである。実機で左右モータの回転方向を実測し、**(a) 配線都合の意図的な補正**なのか、**(b) 実装側のバグ**なのかを確定させること。実測結果によっては Motor.cpp の修正が必要になる。

7.6 Servo クラス

Servo — サーボ PWM ドライバ

ファイル: src/drivers/Servo.h, Servo.cpp

継承: IModule

Output フェーズ

概要

MTU2 チャンネル 0 を PWM モード 1 (分周 $P0\phi/16$ 、周期約 16ms) で駆動し、ステアリングサーボの角度を制御する。出力指示は `sys.srv.angleCmd` に**整数の角度 [度]** で与える。

機能一覧

- MTU2 PWM モード 1 によるサーボ駆動
- 角度 → パルス幅カウントの変換
- `maxAngle` による舵角クランプ
- 反映後の値を `sys.srv.angleActual` へ書き戻し

設定 (ServoConfig)

メンバ	実値	説明
<code>pwmCycle</code>	33332	PWM 周期カウント値
<code>center</code>	3090	中心位置のパルス幅カウント値 (約 1.5ms)
<code>handleStep</code>	23	1 度あたりのカウント値
<code>maxAngle</code>	40	舵角の絶対値上限 [度]
<code>pwmPinFmt</code>	0	P4.0 (TIOC0A)

入出力データ (ServoData / `sys.srv`)

メンバ	型	方向	説明
<code>angleCmd</code>	<code>int</code>	Logic → Servo	指示角 $[-40, +40]$ [度]
<code>angleActual</code>	<code>int</code>	Servo → 外部	クランプ後に実際に反映した角度

関数ドキュメント

◆ `init()`

`bool init() override`

P4.0 を第 2 代替機能 (TIOC0A) に設定し、MTU2 のスタンバイを解除してチャンネル 0 を PWM モード 1 で起動する。分周は $P0\phi/16$ (`MTU2TCR_0 = 0x02`)、バッファ転送は TCNT クリア時 (`MTU2BTM_0 = 0x03`) に設定する。起動時のパルス幅は `_config.center` (中央) である。

◆ updateOutput(sys)

```
void updateOutput(SystemData& sys) override
```

sys.srv.angleCmd を内部の setAngle() に渡して PWM へ反映し、クランプ後の角度を sys.srv.angleActual に書き戻す。

◆ getAngle()

```
int getAngle() const
```

内部に保持している現在の角度を返す（デバッグ用）。

◆ setAngle(angle) [private]

```
void setAngle(int angle)
```

角度をクランプしたうえで、バッファレジスタ MTU2TGRD_0 へ書き込む。変換式は次のとおりで、**正の角度でカウント値が減少する**。

$$\text{MTU2TGRD_0} = \text{center} - \text{angle} \times \text{handleStep}$$

ハードウェアマッピング

信号	ピン	レジスタ	説明
サーボ PWM	P4_0 (TIOC0A)	MTU2TGRD_0	パルス幅（立ち下がりエッジ、バッファレジスタ）

⚠ 要実機確認: 角度の符号と旋回方向の対応

実装では正の角度がカウント値の**減少**に対応する（center - angle * handleStep）。Servo.h と Servo.cpp のコメントは「angle: -maxAngle(右) ~ 0(中央) ~ +maxAngle(左)」 「参考プロジェクト handle() に準拠: 正=左」と記述しており、**正の角度で左旋回**という前提である。

一方、本仕様書の旧版は $val = angle \times 23 + CENTER$ （正の角度でカウント値が増加）と記述しており、**符号が逆**であった。

本節は**現在の実装の実態**を記載したものである。実機でサーボに正の角度を与え、車体が実際にどちらへ切れるかを確認し、RunLogic のクランク/レーンチェンジのハンドル指示（正=左を前提としている）と整合しているかを検証すること。食い違っていた場合は左右のクランクが逆方向に曲がるため、走行が成立しない。

7.7 Onboard クラス

Onboard — オンボード LED / スイッチ ドライバ

ファイル: src/drivers/Onboard.h, Onboard.cpp 継承: IModule Input / Output 両フェーズ

概要

GR-PEACH ボード上のフルカラー LED (RGB 3 色) + ユーザー LED (1 個) とユーザースイッチ (1 個) を制御する。本プロジェクトで唯一、**Input と Output の両フェーズに登録される**モジュールである。

- Input フェーズ: `updateInput()` が `P6_0` を読み `sys.ob.sw` へ格納
- Output フェーズ: `updateOutput()` が `sys.ob.*LedCmd` を内部ラッチへ転記し、GPIO へ一括反映

設定 (OnboardConfig)

メンバ	実値	説明
<code>swPinFmt</code>	0	P6_0 (ユーザースイッチ)
<code>ledUserPinFmt</code>	12	P6_12 (ユーザー LED)
<code>ledRedPinFmt</code>	13	P6_13 (赤)
<code>ledGreenPinFmt</code>	14	P6_14 (緑)
<code>ledBluePinFmt</code>	15	P6_15 (青)

入出力データ (OnboardData / `sys.ob`)

メンバ	型	方向	説明
<code>sw</code>	<code>int</code>	Onboard → Logic	1=押下, 0=解放
<code>ledRedCmd</code>	<code>int</code>	Logic → Onboard	0=消灯, 1=点灯
<code>ledGreenCmd</code>	<code>int</code>	Logic → Onboard	同上
<code>ledBlueCmd</code>	<code>int</code>	Logic → Onboard	同上
<code>userLedCmd</code>	<code>int</code>	Logic → Onboard	同上

内部変数

変数名	型	説明
<code>ledState_[4]</code>	<code>int</code>	LED ラッチバッファ。添字は 0=赤, 1=緑, 2=青, 3=ユーザー

関数ドキュメント

◆ `init()`

```
bool init() override
```

ポート 6 の GPIO を初期化する。LED 4 本をポートモード (PMC6 クリア)・出力 (PM6 クリア)・ソフトウェア I/O 制御 (PIPC6 クリア) に設定し、全 LED を消灯 (High 出力) する。スイッチ P6_0 は PIPC6 / PMC6 をクリアして入力 (PM6 セット) とし、PIBC6 の入力バッファを有効化する。

◆ `updateInput(sys)`

```
void updateInput(SystemData& sys) override
```

`sw()` の結果を `sys.ob.sw` へ格納する。

◆ updateOutput(sys)

```
void updateOutput(SystemData& sys) override
```

sys.ob.*LedCmd を内部ラッチ ledState_[] へ転記した後、4 本の LED をまとめて GPIO へ反映する。ledState_[i] が真なら GPIO.P6 の該当ビットをクリア (Low 出力=点灯)、偽ならセット (High 出力=消灯) する。

◆ sw()

```
int sw()
```

ユーザースイッチの状態を即時読み取りで返す。ラッチ方式**ではなく**、呼び出し時に GPIO.PPR6 を直接読む。スイッチは active-low のため、読み値が 0 のときに 1 (押下) を返す。

戻り値: int — 1: 押下, 0: 解放

main() 冒頭の起動モード判定 (S 8.1) で sys.ob.sw がまだ更新されていない時点から呼べるよう、public のまま残している。通常の走行ロジックからは sys.ob.sw を読むこと。

◆ setUserLed(val) / setColorLed(r, g, b) [private]

```
void setUserLed(int val) / void setColorLed(int r, int g, int b)
```

updateOutput() が内部ラッチを更新するために使う。外部からは呼べない (旧仕様では public だった)。LED を操作したい場合は sys.ob.*LedCmd に代入すること。

ハードウェアマッピング

デバイス	ピン	論理	説明
LED Red	P6_13	Low Active	赤色 LED
LED Green	P6_14	Low Active	緑色 LED
LED Blue	P6_15	Low Active	青色 LED
LED User	P6_12	Low Active	ユーザー LED (要実機確認、下記参照)
SW User	P6_0	Low = 押下	プルアップ、GND 接続

⚠ GPIO 注意事項

- スイッチ読み取りには PIBC6 の入力バッファ有効化が必須
- スイッチ・LED とともに PIPC6 をクリアしてソフトウェア I/O 制御にすること (ペリフェラル I/O 制御のままだと GPIO 操作が効かない)
- sw() はラッチ方式**ではなく**、呼び出し時に直接 GPIO を読む

△ 要実機確認: ユーザー LED の論理

実装 (Onboard::updateOutput()) は RGB 3 色とユーザー LED の 4 本すべてを同一ロジックで扱い、点灯時に Low を出力する。init() も 4 本まとめて High (消灯) で初期化する。つまり実装上はユーザー LED も **Low Active** である。

本仕様書の旧版はユーザー LED のみ **High Active** と記述しており、「Low Active (RGB) と High Active (ユーザー LED) を考慮して出力する」と説明していた。実装にはその分岐が存在しない。

本表は**現在の実装の実態**を記載したものである。実機で `sys.ob.userLedCmd = 1` としたときにユーザー LED が実際に点灯するかを確認し、(a) 旧仕様の記述が誤りだったのか、(b) 実装に分岐が必要なのかを確定させること。

7.8 Serial クラス

Serial — シリアル通信ドライバ

ファイル: src/drivers/Serial.h, Serial.cpp

継承: IModule

即時出力 (フェーズに依存しない)

概要

SCIF2 を使用してデバッグ用シリアル通信を行う。他のドライバとは異なりラッチ方式を採用せず、`printf()` / `print()` 呼び出し時に**即時出力 (ポーリング送信)**する。これにより、途中でクラッシュした場合でもそこまでの出力が確実に端末へ届く。

機能一覧

- SCIF2 によるシリアル通信 (GR-PEACH の DAPLink USB シリアル経由)
- 書式付き出力 `printf()` と固定文字列出力 `print()`
- 即時出力 (バッファリングなし、FIFO 空き待ちのポーリング)

設定 (SerialConfig)

メンバ	実値	説明
baud	230400	ボーレート [bps]
txPinFmt	3	P6_3 (TxD2)
rxPinFmt	2	P6_2 (RxD2)
bufferSize	512	vsnprintf 用の内部バッファサイズ

△ ターミナルの設定は 230400bps

旧仕様の 115200bps は誤りである。端末 (TeraTerm 等) を 115200bps に設定すると文字化けする。ボーレートの導出については § 10.7 を参照。

出力データ (SerialData / sys.ser)

メンバ	型	説明
txCount	uint32_t	送信文字の累計 (デバッグ用)

内部変数

変数名	型	説明
buffer_[512]	char	vsnprintf の出力先。ANSI 色付きの 160 文字行を扱うため 512 バイトに拡大している

関数ドキュメント

◆ init()

bool init() override

STBCR4 の bit5 をクリアして SCIF2 のモジュールストップを解除し、P6_3 / P6_2 を Alt7 (TxD2 / RxD2) に設定してから SCIF2 のレジスタを初期化する。FIFO リセット → ボーレート設定 → 1 ビット時間以上の待機 → FIFO リセット解除 → 送受信有効化 (SCSCR = 0x0030) の順で行う。

◆ updateOutput(sys)

void updateOutput(SystemData& sys) override

IModule インターフェースを満たすためのダミー実装。何もしない。出力は printf() / print() の呼び出し時に即座に行われる。

◆ printf(fmt, ...)

void printf(const char* fmt, ...)

書式付き文字列を vsnprintf で buffer_ に展開し、putChar() 経由で 1 文字ずつ即時送信する。

引数: fmt (const char*) — 書式文字列 (printf 互換)

◆ print(str)

void print(const char* str)

書式解析を行わずに文字列をそのまま送信する。固定文字列や事前に組み立て済みの行バッファを出す場合はこちらのほうが速い。

引数: str (const char*) — 送信文字列

◆ putChar(c) [private]

void putChar(char c)

送信 FIFO に空きができるまで SCFDR をポーリングしてから SCFTDR へ 1 文字書き込み、SCFSR の TDFE / TEND をクリアする。

ハードウェアマッピング

信号	ピン	機能	説明
TxD2	P6_3	SCIF2 Alt7	DAPLink USB シリアルへ物理接続
RxD2	P6_2	SCIF2 Alt7	同上

▷ ピン選択の根拠

SCIF2 の代替ピンとしては P3_0 / P3_2 も存在するが、**GR-PEACH の DAPLink 基板には接続されていない**。mbed の PinNames.h が USBTX = P6_3 と定義しているとおおり、P6_3 / P6_2 を使用すること。

7.9 SDCard クラス

SDCard — SD カード SPI ブロックドライバ

ファイル: src/drivers/SDCard.h, SDCard.cpp

継承: IModule

周期処理なし

概要

RSPI2 を用いた SPI 通信で SD カードにアクセスし、**512 バイトのブロック単位**での読み書きを提供する。本クラスはファイルシステムより下位の層であり、**FatFs の diskio 層から呼び出される**。ログのバッファリングや CSV 整形は SDLogger (§ 7.10) の役割である。

▷ レイヤ構成

SDLogger (ログバッファ・CSV 整形) → FatFs (f_open/f_write) → diskio.cpp (ブリッジ) → SDCard (ブロック I/O) → RSPI2 (SPI)

機能一覧

- RSPI2 による SPI 通信と CS の手動制御
- SD カードの初期化シーケンスとカードタイプ判別
- 複数ブロックの読み書き (readBlocks / writeBlocks)
- カード検出ピンによる挿入判定、CSD からのセクタ数取得

設定 (SDCardConfig)

メンバ	実値	説明
csPinFmt	4	P8_4 (CS, GPIO 手動制御)
cdPinFmt	8	P7_8 (CD, カード検出)
mosiPinFmt	5	P8_5
misoPinFmt	6	P8_6
sckPinFmt	3	P8_3

カードタイプ (CardType)

列挙子	値	説明
CARD_NONE	0	未検出 / 初期化失敗
CARD_V1	1	SD v1.x (標準容量)
CARD_V2_SC	2	SD v2.x 標準容量
CARD_V2_HC	3	SDHC / SDXC (高容量)

内部変数

変数名	型	説明
initialized_	bool	初期化成功フラグ
cardType_	CardType	判別したカードタイプ
sectorCount_	uint32_t	CSD から算出したセクタ数

内部定数

定数名	値	説明
CMD_TIMEOUT	5000	コマンド応答待ちのリトライ上限

関数ドキュメント

◆ init()

bool init() override

RSPi2 を初期化し、SD カードの初期化シーケンス **CMD0** → **CMD8** → **ACMD41** → **CMD58** → **CMD16** を実行する。CMD58 で OCR を読んで SDHC / 標準容量を判別し、CMD16 でブロック長を 512 バイトに固定する。成功時に initialized_ を true にする。

◆ updateOutput(sys)

void updateOutput(SystemData& sys) override

未使用。周期処理は行わない。

◆ readBlocks(buf, block, count)

```
int readBlocks(uint8_t* buf, uint32_t block, uint32_t count)
```

block 番から count ブロック (512 バイト単位) 読み出す。

戻り値: int — 0=成功, 非 0=エラー

◆ writeBlocks(buf, block, count)

```
int writeBlocks(const uint8_t* buf, uint32_t block, uint32_t count)
```

block 番から count ブロック書き込む。

戻り値: int — 0=成功, 非 0=エラー

◆ isReady() / getCardType() / getSectorCount()

```
bool isReady() const / CardType getCardType() const / uint32_t getSectorCount() const
```

初期化状態・カードタイプ・セクタ数をそれぞれ返す。

◆ 主な private メソッド

```
void initSPI() / void setSPIClock(uint32_t hz) / uint8_t spiTransfer(uint8_t data) /  
void csLow() / void csHigh() / bool isCardInserted() / int sendCommand(uint8_t cmd,  
uint32_t arg) / int sendAppCommand(uint8_t cmd, uint32_t arg) / int readData(uint8_t*  
buf, uint32_t len) / int writeData(const uint8_t* buf, uint8_t token) / void  
waitReady()
```

sendCommand() は R1 レスポンスを返し、タイムアウト時は -1 を返す。waitReady() は**戻り値を持たない** (ビジー解除まで待つだけ)。isCardInserted() は CD ピンが Low のときカードありと判定する。

ハードウェアマッピング

信号	ピン	機能	説明
MOSI	P8_5	RSPI2 MOSI (Function 3)	
MISO	P8_6	RSPI2 MISO (Function 3)	
CLK	P8_3	RSPI2 RSPCK (Function 3)	
CS	P8_4	GPIO 出力	手動制御
CD	P7_8	GPIO 入力	カード検出 (Low = 挿入)

7.10 SDLogger クラス

SDLogger — 走行ログ記録モジュール

ファイル: src/drivers/SDLogger.h, SDLogger.cpp

継承: IModule

Output フェーズ (メインループ実行)

概要

走行中のデータを RAM 上のリングバッファに蓄積し、走行終了後に CSV ファイルとして SD カードへ一括書き出す。書き込み先は /data0000.csv ~ /data9999.csv で、連番は renban.txt で管理する。

△ ISR ではなくメインループから呼ぶ

SD カードのバス処理と FatFs にはタイミング制約があるため、本モジュールは s_outputModules[] 配列には登録せず、メインループから毎周期 g_sdlogger.updateOutput(g_sys) を呼ぶ。

機能一覧

- 走行中のログ自動記録（パターン番号による記録開始/終了の自動判定）
- 4000 エントリ（約 2.7MB）のログバッファ
- 各エントリに画像 1 行分（160 画素）の生データを同梱
- CSV 形式でのバッチ書き出しと連番ファイル管理

設定 (SDLoggerConfig)

メンバ	実値	説明
maxEntries	4000	バッファのエントリ数上限（配列上限は SDLOG_MAX_ENTRIES）
imageWidth	160	画像 1 行記録時の幅
recordImageRow	45	画像を記録する行番号。偏差の記録行も兼ねる
patternMinForLog	11	この番号以上のパターンで記録を開始（RP_TRACE_NORMAL）
finishPattern	101	このパターンは記録対象外（RP_FINISH）

入出力データ (SDLoggerData / sys.sd)

メンバ	型	方向	説明
ready	bool	SDLogger → 外部	FatFs マウント成功フラグ
logCount	uint32_t	SDLogger → 外部	現在の記録エントリ数
full	bool	SDLogger → 外部	バッファ満杯フラグ
saveRequested	bool	外部 → SDLogger	true で書き出しを要求
saveDone	bool	SDLogger → 外部	書き出し完了フラグ

ログエントリ (SDLOG_T)

メンバ	型	記録元
cnt_msdwritetime	unsigned int	g_timer_1ms (タイムスタンプ)
pattern	unsigned int	sys.run.pattern
convertBCD	unsigned int	sys.line.sensorBin
handle	signed int	sys.run.handleVal
hennsa	signed int	sys.line.deviation[recordImageRow]
encoder	unsigned int	sys.enc.totalCount
motorL / motorR	signed int	sys.mot.leftActual / rightActual
flagL / flagK / flagR / flagS	signed int	sys.line.leftLine / crossLine / rightLine / centerLine
total	signed int	g_timer_1ms
imageData0[160]	signed int	sys.cam.imageBufferPtr の recordImageRow 行

関数ドキュメント

◆ init()

bool init() override

SDCard を初期化して FatFs をマウントし、ログバッファをリセットする。成功時に mounted_ を true にする。

◆ updateOutput(sys)

void updateOutput(SystemData& sys) override

次の 3 つを毎周期実行する。

- sys.sd.ready / logCount / full を現在値で更新
- sys.run.pattern が patternMinForLog 以上かつ finishPattern 以外なら、SystemData から 1 エントリ分を収集して commit()
- sys.sd.saveRequested が立っていれば saveToSD() を実行し、成功時に sys.sd.saveDone を立てる

◆ saveToSD()

int saveToSD()

バッファ内の全エントリを CSV ファイルとして SD カードへ書き出す。renban.txt から連番を読み、/dataNNNN.csv を作成して書き込み、連番を更新する。

戻り値: int — 0=成功, 非 0=エラー

◆ `current()` / `commit()`

`SDLOG_T& current()` / `void commit()`

`current()` は次に書き込むエントリへの参照を返し、`commit()` が記録番地を 1 つ進める。バッファが満杯の場合、`current()` は最終エントリを返して上書きを防ぐ。

◆ `isReady()` / `getLogCount()` / `isFull()` / `resetLog()`

`bool isReady()` `const` / `unsigned int getLogCount()` `const` / `bool isFull()` `const` / `void resetLog()`

マウント状態・記録数・満杯判定の取得と、バッファのリセット。

使用例

```

1 // main 起動時 (GIC 初期化の後、Camera 初期化の前)
2 g_sdlogger.init();
3
4 // メインループ内
5 g_sdlogger.updateOutput(g_sys); // 走行中ログの自動記録
6
7 if (g_sys.run.finished && !savedToSD) {
8     savedToSD = true;
9     g_sys.mot.leftCmd = 0;
10    g_sys.mot.rightCmd = 0;
11    g_sys.srv.angleCmd = 0;
12    g_sys.sd.saveRequested = true;
13    g_sdlogger.updateOutput(g_sys); // 即座に保存実行
14 }
```

Listing 5: SDLogger の組み込み例 (メインループ)

7.11 RunLogic — 走行ロジック

RunLogic — 走行状態機械 (Logic レイヤー)

ファイル: `src/logic/RunLogic.h`, `RunLogic.cpp`

種別: 自由関数の集合 (クラスではない)

Logic フェーズ

概要

参考プロジェクト `mcr_shiozawa_cclass_2.38m-s/main.cpp` の `intTimer()` 内 `switch(pattern)` 状態遷移を移植したもの。旧設計ではこれを `RunController` というクラスにしていたが、EMA 化にあたりクラスを廃して純関数 `applyDrivingPattern()` に分解し、インスタンス変数はすべて `SystemData::run (RunData)` へ移した。

- 入力: sys.line.* (ライン検出結果)、sys.ob.sw (SW 状態)、sys.run.* (自身の状態)
- 出力: sys.mot.*Cmd, sys.srv.angleCmd, sys.ob.*LedCmd, sys.run.*
- **ハードウェアを直接呼ばない** (g_motor / g_servo / g_onboard を参照しない)
- 全パラメータは RUN_CONFIG / MOTOR_CONFIG から取得する

オリジナルは「エンコーダ累積パルス」で距離を管理していたが、本実装では読みやすさを優先してすべて「経過時間 [ms]」で管理する。時間管理は ModuleTimer (§ 4.4) が担う。

状態データ (RunData / sys.run)

メンバ	型	説明
pattern	int	現在の走行パターン番号
prevPattern	int	直前のパターン (遷移検知用)
handleVal	int	直近のハンドル指示値 [度]
traceOffset	int	ハンドル計算のオフセット
leftMotor / rightMotor	int	calcMotorVal() の結果
leftBrake / rightBrake	int	calcBrakeMotorVal() の結果
lastHandle101	int	FINISH 時に維持するハンドル値
finished	bool	走行終了フラグ。メインループが SD 保存のトリガに使う
stateTimer	ModuleTimer	現パターンに入ってから経過時間 (旧 stateMs_)
totalTimer	ModuleTimer	走行スタートからの経過時間 (旧 totalMs_)
auxTimer	ModuleTimer	補助タイマー (バー検知等、旧 cnt1Ms_)

状態遷移 (RunPattern)

番号	列挙子	役割
0	RP_WAIT_SW	スイッチ入力待ち
1	RP_APPROACH_BAR	スタートバーへ前進
2	RP_WAIT_AT_BAR	バー手前で停止
3	RP_ACCEL_AFTER_BAR	バー上昇後の加速
4	RP_CHECK_BAR_OPEN	バー開放確認
10	RP_WAIT_BAR_OPEN	バー開放待ち
11	RP_TRACE_NORMAL	通常トレース
21	RP_CROSSLINE_DETECTED	クロスライン検出
22	RP_CROSSLINE_SKIP	クロスラインの読み飛ばし
23	RP_CROSSLINE_AFTER	クロスライン後のトレース・クランク検出
30	RP_LEFT_CRANK_ENTRY	左クランク開始
31	RP_LEFT_CRANK	左クランク中
40	RP_RIGHT_CRANK_ENTRY	右クランク開始
41	RP_RIGHT_CRANK	右クランク中
51	RP_RIGHT_HALF_DETECTED	右ハーフライン検出
52	RP_RIGHT_HALF_SKIP	右ハーフラインの読み飛ばし

53	RP_RIGHT_HALF_AFTER	右ハーフライン後のトレース
54	RP_RIGHT_LANE_PREPARE	右レーンチェンジ準備
55	RP_RIGHT_LANE_TURN	右レーンチェンジ：寄せ
56	RP_RIGHT_LANE_STRAIGHT	右レーンチェンジ：直進
57	RP_RIGHT_LANE_COUNTER	右レーンチェンジ：戻し
61	RP_LEFT_HALF_DETECTED	左ハーフライン検出
62	RP_LEFT_HALF_SKIP	左ハーフラインの読み飛ばし
63	RP_LEFT_HALF_AFTER	左ハーフライン後のトレース
64	RP_LEFT_LANE_PREPARE	左レーンチェンジ準備
65	RP_LEFT_LANE_TURN	左レーンチェンジ：寄せ
66	RP_LEFT_LANE_STRAIGHT	左レーンチェンジ：直進
67	RP_LEFT_LANE_COUNTER	左レーンチェンジ：戻し
101	RP_FINISH	走行終了（ログ保存待ち）

▷ パターン番号は他モジュールからも参照される

LineDetector は pattern == 11 / 1 / 2 で検出パラメータを切り替え、SDLogger は patternMinForLog = 11 以上で記録を開始し finishPattern = 101 を除外する。番号を変更する場合は ProjectConfig.h 側の対応値も合わせること。

関数ドキュメント

◆ runLogicInit(sys)

void runLogicInit(SystemData& sys)

起動時に 1 回だけ呼ぶ初期化。sys.run.* をすべて初期値へ戻し、pattern を RP_WAIT_SW に設定して 3 本の ModuleTimer を起動する。あわせて初期出力指示 (sys.mot.*Cmd, sys.srv.angleCmd, sys.ob.*LedCmd) をすべて 0 にクリアする。

◆ applyDrivingPattern(sys)

void applyDrivingPattern(SystemData& sys)

Logic フェーズで毎周期呼ぶ本体。処理の順序は次のとおり。

- 1. sys.ob.userLedCmd を中央 2 点センサ (sensorBin & 0x18) の反応で更新
- 2. calcHandle(sys) でハンドル指示値を算出
- 3. calcMotorVal(sys) で通常走行時のモータ値を算出
- 4. switch (sys.run.pattern) で 29 個の runXxx() 関数へ分岐 (未定義の番号は RP_WAIT_SW へ戻す)
- 5. sys.run.prevPattern を更新

ハンドル指示値の計算

calcHandle() (RunLogic.cpp 内の static 関数) は PID ではなく **比例ゲインの 3 段切替**でハンドル角を決める。本プロジェクトで最も頻繁に調整するロジックである。

dev = sys.line.deviation[traceRow]、corrected = dev + sys.run.traceOffset として:

条件	ハンドル指示値
pattern < RP_TRACE_NORMAL	更新しない (停止系のため直前値を維持)
pattern != RP_TRACE_NORMAL	corrected × handleGainNonNormal
dev ≤ handleStraightAbsThreshold	0 (直線とみなす)
dev ≤ handleGentleAbsThreshold	corrected × handleGainGentle (緩カーブ)
上記以外	corrected × handleGainSharp (急カーブ)

通常走行時のモータ出力は calcMotorVal() が左右とも MOTOR_CONFIG.maxPower に設定する。ブレーキ時は calcBrakeMotorVal() が RUN_CONFIG.brakeTargetMotor を用いる。

距離 → タイマー置換の対応表

参考プロジェクト（エンコーダ距離）→ 本実装（経過時間 ms）の対応:

参考プロジェクト	本実装	備考
encoder.clear()	sys.run.stateTimer.setTime()	changePattern が 状 態 遷 移 時 に 自 動 実 行
encoder.getTotalCount() >= N	sys.run.stateTimer.isExpired(N)	1 pulse ≈ 1ms 換 算
encoder.getCourseCount() >= ONEM*58	sys.run.totalTimer.isExpired(tCourseTimeoutMs)	既定 120 秒
encoder.getCnt() <= lowSpeedLimit	(削除)	速度 判 定 は カ ッ ト

使用例

```
1 // main 起動時（全ドライバの init() 完了後、initOSTMO() の前）
2 runLogicInit(g_sys);
3
4 // OSTMO 1ms 割り込みコールバック内— Logic フェーズ
5 if (!s_debugMode) {
6     applyDrivingPattern(g_sys); // 状態遷移を 1ms 毎に進める
```

```
7 }
8
9 // メインループ走行終了の検知
10 if (g_sys.run.finished && !savedToSD) {
11     savedToSD = true;
12     g_sys.sd.saveRequested = true;
13     g_sdlogger.updateOutput(g_sys);
14 }
```

Listing 6: RunLogic の組み込み例

▶ 旧 RunController からの移行対応

旧仕様書に記載されていた API は次のように置き換わっている。

旧 API	現在の参照方法
g_runController.init()	runLogicInit(g_sys)
g_runController.update()	applyDrivingPattern(g_sys)
isFinished()	g_sys.run.finished を直読
getPattern()	g_sys.run.pattern を直読
getHandleVal()	g_sys.run.handleVal を直読
forceStop()	該当なし。g_sys.run.pattern = RP_FINISH 等で代替

8 実行モデル詳細

8.1 起動シーケンス

main() の初期化順序には**守らないとハングする制約**が複数ある。順序を変更する場合は下表の理由を必ず確認すること。

#	処理	順序の理由
1	runGlobalConstructors()	最初に実行必須 。start.S が__libc_init_array() を呼ばないため、これを飛ばすと全モジュールの_config が BSS のゼロのままになる (§ 10.3)
2	SystemInit()	FPU / MMU / L1 キャッシュを有効化する。この関数の中ではシリアル出力を行わないこと
3	g_onboard.init()	起動モード判定に sw() を使うため、シリアルより先に GPIO を立ち上げる
4	g_serial.init()	以降の初期化ログを出力するため早期に初期化する
5	起動モード判定	g_onboard.sw() の結果でデバッグモード / 走行モードを決める。カメラ初期化前なので結果をシリアルで確認できる
6	initGIC()	Camera より前に実行必須 。VDC5 割り込みの登録が GIC に依存する (§ 10.5)

#	処理	順序の理由
7	g_sdlogger.init()	GIC の後・Camera の前。SPI 通信自体に割り込みは不要だが、カメラより先に済ませて起動時間を短縮する
8	g_camera.init()	VDC5 / DVDEC を初期化しキャプチャを開始する
9	g_motor.init()	MTU2 ch3/4 のスタンバイ解除と PWM 設定
10	g_servo.init()	MTU2 ch0 のスタンバイ解除と PWM 設定
11	g_lineDetector.init()	偏差配列とフラグのクリア
12	runLogicInit(g_sys)	sys.run.* と初期出力指示のクリア
13	initOSTM0()	必ず最後 。下の注意ボックスを参照
14	動作モード別ループへ分岐	runDebugLoop() または runMainLoop()

△ initOSTM0() は必ず最後に呼ぶ

1ms 割り込みが動き出すと、ISR の Output フェーズが**毎 ms Motor と Servo を叩く**。g_motor.init() / g_servo.init() より前に OSTM0 を起動すると、MTU2 がスタンバイ状態のまま PWM レジスタへの書き込みが発生し、サーボ挙動の異常やバス例外によるハングを招く。

この制約は EMA 化によって「Logic だけでなく Output フェーズも常時実行される」構造に変わったことで新たに生じたものである。

△ GIC は Camera より前に初期化する

逆順にすると、カメラ初期化中に登録された VDC5 のレベルトリガ割り込みが GIC 無効のため発火せずに蓄積し、後から CPSIE i を実行した瞬間に一齐発火して**割り込みストーム**になる。詳細は [S 10.5](#) を参照。

8.2 3 フェーズ割り込みハンドラ

generate/inthandler.c の INT_Excep_OSTMIO() から ostm0_interrupt_callback() が呼ばれる。この関数が実質的なシステムの心臓部である。

```
1 void ostm0_interrupt_callback(void)
2 {
3     g_timer_1ms++;      // ModuleTimer の時刻源
4     g_cnt_printf++;     // シリアル表示の間隔カウンタ
5
6     // ===== Input フェーズ =====
7     // ハードウェア -> SystemData
8     for (int i = 0; i < N_IN; ++i) {
9         if (s_inputModules[i]->enabled) s_inputModules[i]->updateInput(g_sys);
10    }
11
12    // ===== Logic フェーズ =====
13    // SystemData -> SystemData
14    if (!s_debugMode) {
15        applyDrivingPattern(g_sys);
16    } else {
17        // デバッグモードでも USER_LED は中央点センサ反応で点灯させる
18        g_sys.ob.userLedCmd = (g_sys.line.sensorBin & 0x18) ? 1 : 0;
19    }
```

```
20
21 // ===== Output フェーズ =====
22 // SystemData -> ハードウェア
23 for (int i = 0; i < N_OUT; ++i) {
24     if (s_outputModules[i]->enabled) s_outputModules[i]->updateOutput(g_sys);
25 }
26 }
```

Listing 7: 1ms 割り込みコールバック (mcr_camera_2026base.cpp)

配列	登録モジュール	備考
s_inputModules[]	g_onboard, g_camera	g_encoder は現状未登録。g_lineDetector は処理が重い ためメインループ側
s_outputModules[]	g_motor, g_servo, g_onboard	g_sdlogger はバス制約のためメインループ側。 g_serial は即時出力のため登録不要

▷ Onboard は両方の配列に登録される

g_onboard は Input (SW 読み取り) と Output (LED 反映) の両方を持つため、2 つの配列の両方に登場する。同一インスタンスへのポインタである。

⚠ 1ms を超える処理が入る

XIP 環境では Camera::updateInput() の 1 ステップが約 8ms かかるため、実際の割り込み間隔は 1ms を守れず、g_timer_1ms は実時間より遅く進む (§ 10.1)。フレーム処理が完了して Vfield 待ちに入る期間にメインループへ CPU 時間が返る構造で成立させている。時間精度が要求される処理をこの ISR に足さないこと。

8.3 メインループと動作モード

起動時のスイッチ状態で 2 つのモードに分岐する。判定は main() の早い段階 (カメラ初期化前) で行うため、モード選択の結果をシリアルで確認できる。

起動時 SW	モード	ループ関数と動作
押されていない	走行モード	runMainLoop()。Logic フェーズが有効になり、ライン検出・ログ記録・走行終了時の SD 保存を行う
押されている	デバッグモード	runDebugLoop()。Logic フェーズをスキップし、カメラ画像を 0/1 のパターンでシリアルへ描画する

8.3.1 走行モード (runMainLoop)

- 1. sys.cam.frameReady が立っていれば false に戻し、g_lineDetector.updateInput(g_sys) を実行する
- 2. g_sdlogger.updateOutput(g_sys) を毎周期呼ぶ (走行中ログの自動記録)
- 3. sys.run.finished が立ったら 1 回だけモータ・サーボを停止させ、sys.sd.saveRequested = true として SD へ書き出す
- 4. g_cnt_printf が DEBUG_PRINT_INTERVAL_MS(200) に達したらステータス行 (タイマー・フレーム数・パターン・ハンドル・各ラインフラグ・偏差・ログ数) を出力する

8.3.2 デバッグモード (runDebugLoop)

1. フレーム完了時に `g_lineDetector.updateInput(g_sys)` を実行する (走行モードと同じ)
2. 200ms ごとに、閾値 `DEBUG_THRESHOLD(170)` で 2 値化した 30-100 行 (2 行飛ばし) を ANSI エスケープ付きで描画する
3. 偏差が示すライン中心位置を黄色背景でハイライトする

▷ シリアル出力の最適化

1 ピクセルごとに `printf()` を呼ぶと `vsnprintf` のオーバーヘッドで 1 フレームあたり数千回の書式解析が発生し、描画が極端に遅くなる。現在の実装は **1 行分を静的バッファ `lineBuf[]` に組み立ててから `print()` で一括送信**する方式に変更済みである。固定のヘッダ行も `print()` を使い、書式解析自体を回避している。

8.4 出力反映のデータフロー

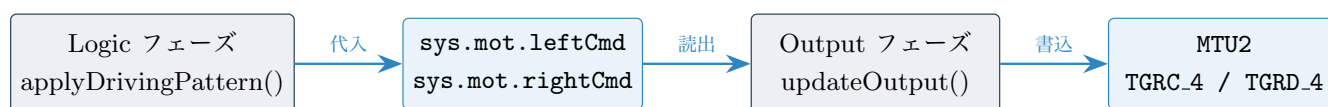


図 2: Motor を例にした出力反映のデータフロー

Logic フェーズはドライバのメソッドを一切呼ばず、`SystemData` の `*Cmd` メンバに**代入するだけ**である。実際のレジスタ書き込みは Output フェーズの `updateOutput()` でのみ発生する。この分離により、Logic の途中経過が物理出力へ漏れることがなくなり、Logic フェーズだけを PC 上でテストできる。

9 パラメータ調整ガイド

本章は `src/core/ProjectConfig.h` の実値を一覧化したものである。**走行調整は原則このファイルだけを編集する。**

△ 値の正本は ProjectConfig.h

`RunLogic.h` や各ドライバヘッダの構造体宣言にはコメントで数値が併記されており、現在は `ProjectConfig.h` の実値と一致している。ただしこれは**あくまで参考であり、正本は常に `ProjectConfig.h` 側**である。

過去に片方だけを更新して `RunLogicConfig` の 31 項目中 23 項目が食い違った実績があるため、`ProjectConfig.h` の値を変更したときは**ヘッダ側のコメントも必ず同時に更新**すること。

▷ C++17 の inline const を使用

`ProjectConfig.h` は `inline const` でインスタンスを定義しているため、**C++17 以上でのコンパイルが必要**である。e2 studio のコンパイラ指定が古い場合は、`extern const` 宣言 + `ProjectConfig.cpp` での実体定義に切り替えること。

9.1 RUN_CONFIG — 走行ロジック

9.1.1 状態タイマー定数 [ms]

メンバ	実値	説明
tLineSkipMs	100	クロス/ハーフラインの通過（読み飛ばし）時間
tCrankMs	250	クランク旋回時間
tLaneTurnMs	250	レーンチェンジ：寄せ時間
tLaneStraightMs	0	レーンチェンジ：直進時間
tLaneCounterMs	250	レーンチェンジ：戻し時間
tCourseTimeoutMs	120000	コース 1 周想定のタイムアウト（120 秒）
tBarWaitPreMs	100	pattern 1 で前進開始までの待機
tBarWaitPostMs	100	pattern 2 で次へ進むまでの待機
tAfterBarGoMs	100	pattern 3 でバー後に加速する時間
tHalfAfterTimeoutMs	100	ハーフライン後のタイムアウト
tTraceLineEnableMs	100	通常トレースでライン検出を有効化する経過時間
tBarApproachWaitMs	100	pattern 2 でハンドル制御を行うアシスト時間

9.1.2 走行パラメータ

メンバ	実値	説明
crankHandle	30	クランク旋回角度 [度]
crankMotorOut	70	クランク外輪のモータ出力 [%]
crankMotorIn	70	クランク内輪のモータ出力 [%]
laneHandle	25	レーンチェンジ寄せ角度 [度]
laneCounterHandle	25	レーンチェンジ戻し角度 [度]
laneMotorIn	60	レーンチェンジ内輪 [%]
laneMotorOut	60	レーンチェンジ外輪 [%]
laneStraightMotor	60	レーンチェンジ直進時のモータ出力 [%]
laneCounterMotorIn	60	レーンチェンジ戻し内輪 [%]
laneCounterMotorOut	60	レーンチェンジ戻し外輪 [%]
brakeTargetMotor	60	ブレーキ走行時の目標モータ出力 [%]
approachMotorPower	60	スタートバーへ前進する際のモータ出力 [%]

9.1.3 検出行

メンバ	実値	説明
traceRow	48	通常トレース時に偏差を取得する行
approachRow	95	スタート時に偏差を取得する行

9.1.4 ハンドル計算ゲイン（最頻調整対象）

メンバ	実値	説明
handleStraightAbsThreshold	0	dev がこの値以下なら直線扱い（ハンドル 0）
handleGentleAbsThreshold	7	dev がこの値以下なら緩カーブ扱い
handleGainGentle	0.24	緩カーブのゲイン
handleGainSharp	0.24	急カーブのゲイン
handleGainNonNormal	0.24	pattern != RP_TRACE_NORMAL 時のゲイン

▷ 調整の入口はここ

ProjectConfig.h のコメントも「特に頻繁に調整するもの」として handleGain* 系、crank* / lane* 系、LINE_DETECTOR_CONFIG.devBrightnessRatio と各 pattern* を挙げている。現状は 3 つのゲインが同値 (0.24) だが、直線・緩カーブ・急カーブで個別に調整できる構造になっている。

9.2 MOTOR_CONFIG / SERVO_CONFIG — アクチュエータ

Config	メンバ	実値	説明
MOTOR_CONFIG	pwmCycle	33332	PWM 周期カウンタ (1ms @ P0φ/1)
	maxPower	70	出力上限 [%]。通常走行速度を直接決める
	leftPwmPinFmt	4	P4.4 (TIOC4A)
	rightPwmPinFmt	5	P4.5 (TIOC4B)
	leftDirPinFmt	6	P4.6
	rightDirPinFmt	7	P4.7
SERVO_CONFIG	pwmCycle	33332	PWM 周期カウンタ
	center	3090	中心位置のパルス幅カウンタ (約 1.5ms)
	handleStep	23	1 度あたりのカウンタ値
	maxAngle	40	舵角の絶対値上限 [度]
	pwmPinFmt	0	P4.0 (TIOC0A)

9.3 LINE_DETECTOR_CONFIG — ライン検出

9.3.1 基本設定と偏差計算パラメータ

メンバ	実値	説明
detectRowDefault	57	検出行の既定値
detectHeightDefault	3	検出高さの既定値
devBrightnessRatio	0.87	偏差計算に使う最大輝度比（頻出の調整対象）
devMinasDiffTh	-8	偏差の負側の差分閾値
devPlusDiffTh	8	偏差の正側の差分閾値
devOutlierThY	5	外れ値判定に使う y 方向の閾値
devLeftDefaultX	70	左側の既定 x 座標
devRightDefaultX	90	右側の既定 x 座標
devBottomCenterOffset	10	最下行の中心オフセット

9.3.2 パターン別パラメータ

メンバ	Normal	Start	Other	説明
crosslineWidth	70	60	-1	-1 は実行時に detectRow_ + 20 を算出
detectHeight	3	10	3	検出に使う行の厚み
crossCountThreshold	-1	59	-10	負値は crosslineWidth との加算値
brightnessRatio	0.69	0.0	0.68	最大輝度に対する相対閾値
brightnessAbs	0	90	0	> 0 なら絶対閾値を優先
centerWidth	45	100	45	センターライン判定幅

適用条件は Normal が pattern == 11、Start が pattern == 1 または 2、Other がそれ以外である。

9.3.3 センターライン検出・8点センサ

メンバ	実値	説明
centerRowNum	47	センターライン判定に使う行数
centerCountThreshold	10	センターライン判定の画素数閾値
centerBrightnessAbs	170	センターライン判定の絶対輝度閾値
sensorBinRow	60	8点センサを取得する行
sensorBinThreshold	180	8点センサの閾値
sensorBinDiff	8	8点センサの差分閾値

9.4 その他の Config

Config	メンバ	実値	説明
CAMERA_CONFIG	pixelWidth	160	水平ピクセル数
	pixelHeight	120	垂直ピクセル数
	thresholdDefault	170	8点センサのデフォルト閾値
	thresholdRow	60	8点センサ取得行
	thresholdDiff	8	8点センサ閾値の差分
ONBOARD_CONFIG	swPinFmt	0	P6_0
	ledUserPinFmt	12	P6_12
	ledRedPinFmt	13	P6_13
	ledGreenPinFmt	14	P6_14
	ledBluePinFmt	15	P6_15
ENCODER_CONFIG	filterN	4	移動平均の段数
	rcAlpha	0.5	RC フィルタ係数
	aPinFmt	0	P1_0 (TCLKA)
	bPinFmt	10	P1_10 (TCLKB)
SERIAL_CONFIG	baud	230400	ボーレート [bps]
	txPinFmt	3	P6_3 (TxD2)
	rxPinFmt	2	P6_2 (RxD2)
	bufferSize	512	vsnprintf 用バッファ

Config	メンバ	実値	説明
SDCARD_CONFIG	csPinFmt	4	P8.4 (CS)
	cdPinFmt	8	P7.8 (CD)
	mosiPinFmt	5	P8.5
	misoPinFmt	6	P8.6
	sckPinFmt	3	P8.3
SDLOGGER_CONFIG	maxEntries	4000	ログバッファのエントリ数
	imageWidth	160	画像 1 行記録時の幅
	recordImageRow	45	画像・偏差を記録する行
	patternMinForLog	11	記録開始パターン (RP_TRACE_NORMAL)
	finishPattern	101	記録対象外パターン (RP_FINISH)

▷ SDCardConfig だけ include 経路が違う

SDCard には {Module}Data が存在しないため、SystemData.h は SDCard.h を include しない。そのため ProjectConfig.h が SDCard.h を個別に include して SDCardConfig 型を取得している。

10 ベアメタル固有の実装知見

本章は mbed OS が肩代わりしていた処理を自前で用意するにあたって判明した、このプロジェクト固有の技術的な前提をまとめる。いずれも **知らずに触ると起動しなくなる類のもの** である。

10.1 XIP 実行と命令フェッチ制約

本プロジェクトは SPI フラッシュ (0x1800xxxx) 上のコードを RAM へ展開せずに直接実行する XIP モードで動作する。

SPIBSC (SPI バスコントローラ) のプリフェッチバッファは 32 バイトしかなく、**ループの後方分岐でバッファが無効化される**ため、ループの反復ごとに SPI フラッシュから命令を再フェッチすることになる。その結果、Camera::imageCopy() の内側ループ (9600 バイトのコピー) だけで推定 7-8ms を要し、OSTM0 の 1ms 周期を大幅に超過する。

⚠ 割り込み内に重いループを置かない

1ms 割り込みの中で 1ms を超える処理を行うと、コールバック実行中に次の割り込みが保留され、コールバック終了と同時に再発火する。CPU が 100% 割り込み処理に占有され、メインループが一切実行されなくなる。

現在は Camera のフレーム処理を 4 ステップに分割し (§ 7.3)、処理完了後の Vfield 待ち期間にメインループへ CPU 時間を返すことで成立させている。

10.2 MMU / L1 キャッシュの有効化

src/system/system_init.c の SystemInit() は mbed OS の system_RZ_A1H.c を移植したもので、main() の冒頭で呼ぶ。

- 1. __FPU_Enable() — VFPv3 有効化 (FPEXC.EN = 1)

2. CPG.SYSCR3 = 0x0F — SRAM 全バンクの書き込み許可
3. TLB / 分岐予測配列 / 命令キャッシュの無効化とデータキャッシュの全無効化
4. MMU_CreateTranslationTable() (mmu_rzA1H.c)
5. MMU_Enable() — SCTLR.M = 1, SCTLR.AFE = 1
6. L1C_EnableCaches() — I-cache + D-cache 有効化
7. L1C_EnableBTAC() — 分岐ターゲットアドレスキャッシュ有効化

mbed OS が行っていた RZ_A1_InitClock() / RZ_A1_InitBus() はブートローダーが設定済みのため省略し、GIC の初期化は initGIC() で別途行う。L2 キャッシュは未使用である。

△ SystemInit() の中でシリアル出力をしないこと

キャッシュと MMU の切り替え中にペリフェラルへアクセスすると挙動が不定になる。初期化ログは SystemInit() が戻ってから出力する。

▷ XIP の制約はキャッシュ有効化後も残っている

L1 命令キャッシュが有効になれば、原理的には 2 回目以降のループ反復はキャッシュから供給されるはずである。しかし現在も Camera の 4 ステップ分割が必要な状態であることから、§ 10.1 の制約は実効的に解消していないと考えられる。**実際の処理時間を実機で計測して見直す余地がある。**

10.3 グローバル C++ コンストラクタの手動実行

generate/start.S は HardwareSetup() の後に main() を直接呼び、__libc_init_array() を呼ばない。一方 GCC 13 はグローバルコンストラクタを .init_array セクションへ出力するため、何もしなければ**グローバル ctor が一切実行されない**。

対策として次の 2 点を実装している。

1. generate/linker.script.ld に .init_array の収集と __init_array_start / __init_array_end ラベルの定義を追加
2. main() の冒頭で runGlobalConstructors() を呼び、両ラベル間のポインタを手動で反復実行

```

1 extern "C" {
2     typedef void (*ctor_fn)(void);
3     extern ctor_fn __init_array_start[];
4     extern ctor_fn __init_array_end[];
5 }
6
7 static void runGlobalConstructors(void)
8 {
9     for (ctor_fn *p = __init_array_start; p < __init_array_end; ++p) {
10         (*p)();
11     }
12 }
```

Listing 8: グローバルコンストラクタの手動実行

△ EMA 化で初めて顕在化した不具合

EMA 化以前は各ドライバが static const しか使っておらず、ctor の中身が空でも問題にならなかったため症状が出なかった。Config 注入で _config メンバが増えたことにより、「ctor が走らない → _config が

BSS ゼロのまま」となり、Servo の PWM 出力が 0（ペリフェラルのロック）、LineDetector の全閾値が 0（検出の全崩壊）といった致命的な不具合になった。

新しいグローバルインスタンスを追加し、内部状態を ctor 初期化リストに置いた時点でこの仕組みに依存することになる点を忘れないこと。

10.4 ベアメタル向けダミーシンボル

グローバル ctor を有効化すると GCC が以下のシンボルを参照するようになるが、ベアメタル環境（newlib stubs 不使用・dynamic loader 無し）ではいずれも意味を持たない。リンクを通すためダミーで定義している。

シンボル	扱い
__dso_handle	__cxa_atexit() 用の DSO ハンドル。単一実行像のため任意の固定値でよい
__fini	__libc_fini_array() が呼ぶ関数。本プロジェクトは exit に到達しないため空実装
__cxa_atexit	静的デストラクタの登録。常に成功扱い（0 を返す）で何もしない

10.5 GIC 割り込みディスパッチと VBAR

10.5.1 IRQ ディスパッチャの実装

generate/inthandler.c の INT_Except_IRQ() は生成時点では空関数であり、そのままでは割り込みが発生してもハンドラが実行されず、GIC 側の割り込み要求もクリアされない。Cortex-A9 向けの正しいディスパッチロジックを手動で追加している。

- 1. INTC.ICCIAR を読んで割り込み要因 ID を取得する
- 2. 動的 IRQ ハンドラテーブル g_irq_handlers[256] から該当ハンドラを呼ぶ
- 3. INTC.ICCEOIR へ書き込んで EOI（End of Interrupt）を通知する

10.5.2 VBAR の初期化

Cortex-A9 は「自前の割り込みベクタテーブルがどこにあるか」を VBAR（Vector Base Address Register）で知る。未設定だと割り込み発生時にデフォルトの 0x00000000 付近（BootROM 等）へ飛んでフリーズする。generate/start.S に次の 1 命令を追加して vector_table のアドレスを登録している。

```
mcr p15, 0, r0, c12, c0, 0
```

10.5.3 IRQ モードのスタックポインタ

Cortex-A9 は割り込み時に自動で IRQ モードへ遷移するが、IRQ モード用の SP が初期化されていないと、C 言語のハンドラがローカル変数を使った瞬間にクラッシュする。start.S で 4KB の IRQ 専用スタックを確保している。

10.5.4 GIC グローバル有効化の順序

initGIC() は GIC ディストリビュータ（ICDDCR）、CPU インターフェース（ICCICR）、割り込み優先度マスク（ICCPMR）を有効化し、CPSIE i で CPU の IRQ を許可する。これをカメラ初期化より前に実行しなければならない。

△ 順序を誤ると割り込みストームでハングする

カメラ初期化中に VDC5 割り込み (VSYNC / VFIELD, GIC ID 75-97) がレベルトリガとして GIC に登録されるが、GIC が未有効だと発火しない。このとき WaitVsync(1) / WaitVfield(2) は割り込みではなくタイムアウトで抜けている。

その後に GIC を有効化して CPSIE i を実行した瞬間、蓄積されたレベルトリガ割り込みが一斉に発火する。VDC5 割り込み (優先度 0x28) は OSTM0 (優先度 0x80) より高優先度のため、OSTM0 を常に横取りし続け、CPU が割り込み処理から抜けられなくなる。

mbed OS ではブートコードが GIC / IRQ を先に有効化しているため、この問題は起きない。

10.6 OSTM0 の設定手順

initOSTM0() は 1ms インターバルタイマーを設定する。順序と設定値の両方に落とし穴がある。

手順	内容と注意点
スタンバイ解除	CPG.STBCR5 の bit1 をクリアする
タイマー停止	OSTMnTT = 0x01。OSTMnTE が 0 になるまで待つこと。完全に停止していない状態で OSTMnCMP を書き換えても値が無視される
周期設定	OSTMnCMP = 33333 (P0φ = 33.33MHz で 1ms)
モード設定	OSTMnCTL = 0x01。MD0 = 1 が「コンペアマッチごとに割り込み」。0x00 (MD0 = 0) は「カウント開始時のみ」の単発動作になる
GIC 設定	ICDICER4 / ICDICPR4 / ICDISER4 で OSTM0 を有効化。ICDICFR8 でエッジトリガに設定する (GIC の既定はレベルトリガだが OSTM0 はパルス状のエッジ割り込みを発行する)
優先度 / ターゲット	ICDIPR33 に 0x80、ICDIPTR33 に 0x01
カウント開始	OSTMnTS = 0x01

10.7 SCIF2 ボーレートのクロックソース

SCIF のボーレートジェネレータが使うのは P0φ (33.33MHz) ではなく P1φ (66.67MHz) である。これを取り違えると実効ボーレートが約 2 倍ずれて文字化けする。

$$B = \frac{P1\phi}{8 \times (SCBRR + 1)} = \frac{66666666}{8 \times 36} = 231481 \approx 230400 \text{ bps} \quad (\text{誤差 } 0.47\%)$$

レジスタ	設定値	意味
SCSMR	0x0000	8bit データ / ストップ 1 / パリティなし
SCEMR	0x0081	BGDM = 1(bit7), ABCS = 1(bit0) → 除算比 8
SCBRR	35	mbed の DL = (P1CLK + 4*baud) / (8*baud) - 1 と同一
SCSCR	0x0030	TE = 1, RE = 1 (送受信有効化)

△ STBCR4 は 8 ビットレジスタ

SCIF2 のモジュールストップ解除は STBCR4 の bit 5 である。かつて ~(1 << 14) と書いていた時期があり、uint8_t への代入で 0xFF に切り詰められた結果、モジュールストップが解除されないまま SCIF2 のレジスタにアクセスしてバスエラー / データアボートで全動作が停止していた。mbed の MBRZA1H.h が

CPG_STBCR4_BIT_MSTP45 = 0x20 と定義しているとおり、~(1 << 5) が正しい。

▷ ステータスレジスタは W0C 方式でクリアする

SCFSR は直接代入 (= 0x0000) ではなく、&= ~0x00F3 のように「1 を読んで 0 を書く」方式でクリアすること。

10.8 VDC5 / DVDEC の初期化

10.8.1 DisplayBase API の採用

当初は VDC5 / DVDEC をレジスタ直叩きで初期化していたが、設定値がほぼ 0 のままでカメラが正しく初期化されなかった。実績のある mbed-gr-libs の gr_peach_vdc5.c を取り込み、高レベル API (Graphics_init, Graphics_Video_init, Video_Write_Setting) を呼ぶ方式へ全面的に置き換えている。これに伴い pinmap.h / mbed_assert.h 等の mbed OS 依存ダミーヘッダを src/drivers/video/ に追加している。

10.8.2 LVDS PLL エラー (error = 6) の回避

LCD を使わずカメラ専用で使う構成では、R_VDC5_Initialize がエラーコード 6 (VDC5_ERR_PARAM_EXCEED_RANGE) を返し、Graphics_init が失敗する問題があった。

原因: lcd_type が PARALLEL_RGB (非 LVDS) でも panel_icksel が VDC5_PANEL_ICKSEL_LVDS のまま残り、さらに lvds_pll_calc() が計算した PLL パラメータ (double 型の nfd 等) を uint16_t に変換する際、ベアメタル環境での浮動小数点演算結果が禁止範囲に入り、LVDS PLL フィードバック分周値 (NFD) の範囲チェックに引っかかっていた。

対策: LCD 未使用時は panel_icksel = VDC5_PANEL_ICKSEL_PERI (P1 ペリフェラルクロック) を使用し、init_lvds = NULL として LVDS PLL の初期化自体をスキップする。カメラの NTSC キャプチャに LVDS PLL は不要で、P1 クロックで十分動作する。

10.9 generate/ の手動修正箇所

generate/ は e2 studio の自動生成ディレクトリだが、以下は手動で修正している。コード生成を再実行するとこれらの修正が上書きされるため注意すること。

ファイル	手動修正の内容
inthandler.c	動的 IRQ ハンドラテーブル g_irq_handlers[256] (VDC5 割り込み用) の追加。 INT_Excep_IRQ() に ICCIAR 読み取り → ハンドラ呼び出し → ICCEOIR 書き込みを実装。 INT_Excep_OSTMIO() から ostm0_interrupt_callback() を呼ぶ処理を追加
interrupt_handlers.h	個別ペリフェラル割り込みハンドラから __attribute__((interrupt)) を削除し __attribute__((used)) のみにする (下記の注意ボックスを参照)。上位の基本例外 IRQ / FIQ / SWI は変更しない
linker_script.ld	.init_array の収集と __init_array_start / __init_array_end ラベルの定義を追加
start.S	VBAR の設定と IRQ モード用スタック (4KB) の確保を追加

△ `__attribute__((interrupt))` による二重リターン

ARM Cortex-A のコンパイラは `__attribute__((interrupt))` を付けた関数を、通常の `bx lr` ではなく割り込み専用の戻り命令 `subs pc, lr, #4` でコンパイルする。

本プロジェクトでは**最上位の IRQ ハンドラがすでにこの特殊な戻り処理を行っており**、`INT_Except_OSTMI0()` 等の個別ハンドラは単なる C 関数として呼ばれる。それにもかかわらず属性が付いていたため、普通の C 関数として呼ばれたのに割り込み用の戻り命令を実行してしまい、PC が不正なアドレスへ飛んでクラッシュしていた。

10.10 DevContainer によるドキュメントビルド

`doc/main.tex` のコンパイル環境は `.devcontainer/` に定義してある。VS Code で本プロジェクトを開き「Dev Containers: Reopen in Container」を実行すれば、どの環境でも同一条件でビルドできる。ビルドチェーンは `doc/.latexmkrc` に定義した `uplatex + dvipdfmx` である。

付録 A 不具合と対策の記録

旧仕様書 `mcr_camera_2026base.SPEC.md` に記録されていた修正履歴を、症状から原因へ辿れる形に再構成したものである。同種の症状に遭遇したときの参照先として残す。

A.1 起動・割り込み系

症状	原因	対策
LED も点滅せず割り込みが一切動かない	<code>INT_Except_IRQ()</code> が空実装で、GIC の割り込み要求もクリアされない	ICCIAR 読み取り → ハンドラ呼び出し → <code>ICCEOIR</code> 書き込みを実装 (§ 10.5)
割り込み発生時にフリーズする	<code>VBAR</code> が未設定で、CPU がデフォルトの <code>0x00000000</code> 付近へ飛ぶ	<code>start.S</code> で <code>mcr p15, 0, r0, c12, c0, 0</code> により <code>VBAR</code> を設定
ハンドラ内でローカル変数を使うとクラッシュ	IRQ モードのスタックポインタが未初期化	<code>start.S</code> で 4KB の IRQ 専用スタックを確保
ハンドラから戻る瞬間にクラッシュ	個別ハンドラに <code>__attribute__((interrupt))</code> が付いており、通常の C 関数として呼ばれたのに割り込み用の戻り命令を実行	個別ハンドラの属性を <code>__attribute__((used))</code> のみに変更 (§ 10.9)
割り込み周期が設定値の 10 倍速い	<code>OSTMnTE</code> が 0 になる前に <code>OSTMnCMP</code> を書き換え、値が無視されていた	<code>OSTMnTT</code> で停止要求後、 <code>OSTMnTE == 0</code> をポーリングしてから書き込む (§ 10.6)
割り込みが 1 回しか発生しない	<code>OSTMnCTL = 0x00 (MD0 = 0)</code> はカウント開始時のみの単発動作	<code>OSTMnCTL = 0x01 (MD0 = 1)</code> に修正
GIC がエッジ割り込みを取りこぼす	GIC の既定はレベルトリガだが <code>OSTM0</code> はパルス状のエッジ割り込みを発行する	<code>ICDICFR8</code> で <code>OSTM0 (IRQ134)</code> をエッジトリガに設定

症状	原因	対策
「カメラ初期化完了」の後に完全フリーズ	蓄積された VDC5 レベルトリガ割り込みが CPSIE i で一斉発火し、優先度 0x28 の VDC5 が優先度 0x80 の OSTM0 を横取りし続ける	GIC のグローバル有効化を initGIC() に分離し、カメラ初期化より前に実行 (§ 10.5)
サーボ出力が 0 / ライン検出が全崩壊	start.S が _libc_init_array() を呼ばず、グローバル ctor が実行されないため _config が BSS ゼロのまま	main() 冒頭で runGlobalConstructors() を実行 (§ 10.3)

A.2 シリアル通信系

症状	原因	対策
書き込み後に LED 点滅を含む全動作が停止	STBCR4 は 8bit レジスタなのに ~(1 << 14) を代入し、0xFF に切り詰められてモジュールストップが解除されないままアクセス	~(1 << 5) に修正。SCFSR のクリアも W0C 方式に変更 (§ 10.7)
TeraTerm に何も表示されない	P3_0 / P3_2 は SCIF2 の代替ピンとしては正しいが、DAPLink 基板には物理接続されていない	P6_3 (TxD2) / P6_2 (RxD2) の Alt7 に変更
230400bps 設定で文字化けする	SCEMR = 0x0080(ABCS = 0) + BRR = 8 で実効 $P1\phi/(16 \times 9) = 462963\text{bps}$ となり約 2 倍ずれていた	SCEMR = 0x0081, SCBRR = 35 に修正 (§ 10.7)
シリアル出力が非常に遅い	1 ピクセルごとの printf 呼び出し (1 フレーム 5600 回の vsnprintf)	行単位で静的バッファに組み立て print() で一括出力、ボーレートを 230400 に統一

A.3 カメラ系

症状	原因	対策
Graphics_init が error = 6 を返す	非 LVDS 構成でも panel_icksel が LVDS のまま残り、LVDS PLL の NFD 範囲チェックに引っかかる	panel_icksel = VDC5_PANEL_ICKSEL_PERI かつ init.lvds = NULL (§ 10.8)
カメラが正しく初期化されない	レジスタ直叩き実装の初期化値がほぼ 0 のままだった	mbed-gr-libs の DisplayBase API に全面的に置き換え
カメラ出力がぐちゃぐちゃ	imageCopy が VDC5 の未書き込みバッファを読んでいた	VDC5 の書き込み先バッファを直接読むように変更
同じ画像が表示され続ける	フレーム処理中の Vfield 変化で frameStep_ が 0 にリセットされ続け、extractBrightness に到達しない	処理中 (ステップ 0-3) は Vfield 変化を無視し、完了後にのみ次の Vfield を待つ (§ 7.3)

症状	原因	対策
画像が乱れる	4 ステップの合計が Vfield 間隔 (16.7ms) を超え、imageCopy と extractBrightness が異なるフィールド値を使う	ステップ 0 で capturedField_ にキャプチャし、4 ステップ全体で同じ値を使用
フレーム更新が不安定 / 停止する	カメラ更新をメインループで行うと、シリアル出力中 (約 250ms) にフレーム更新が止まる	g_camera.updateInput() を 1ms 割り込み内へ戻し、処理完了後の Vfield 待ち期間にメインループへ CPU を返す構造にする
「タイマー開始」の後に何も出力されない	XIP 環境で imageCopy が 7-8ms かかり、CPU が 100% 割り込み処理に占有される	フレーム処理を 4 ステップに分割 (S 10.1)